

Series Q Q3

Hands-on II: multi-sample integration and downstream analysis

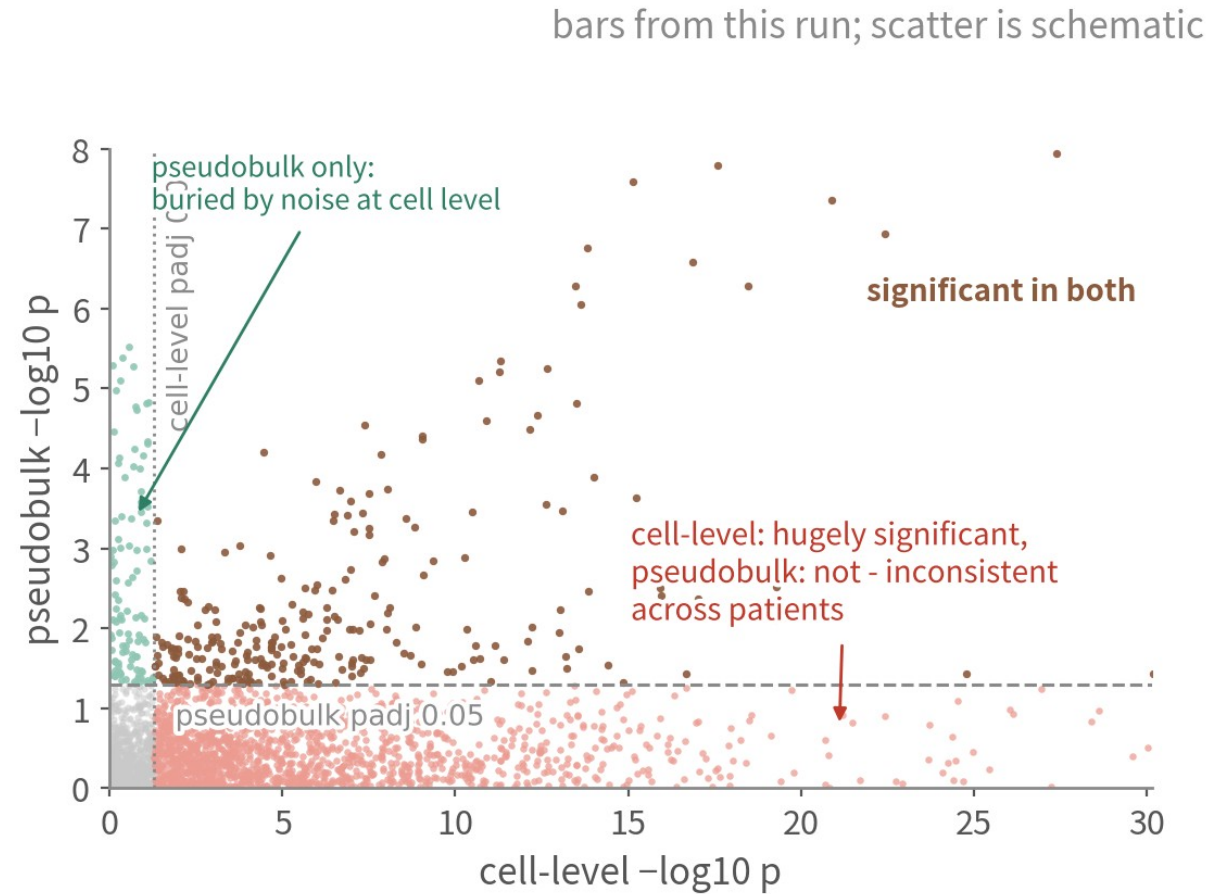
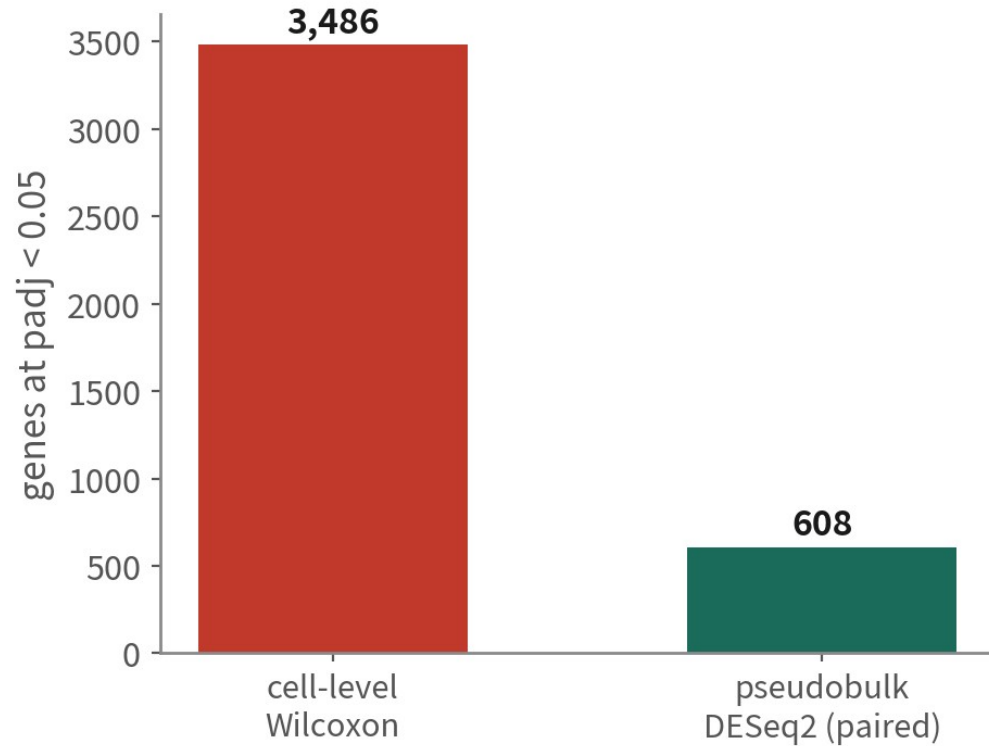
Four GBM patients (core vs infiltrating periphery). Call malignancy — which the single-patient data in Q2 could not support — put DE at the patient level, then choose the downstream route.

About 70 min | Prerequisites: Q2 | Scripts: R/04-10 | Deeper: B11-B17



<https://github.com/Charlene717/scRNA-seq-tutorial-Qseries>

Same question, two answers



Which genes differ between core and periphery within one cell type? Cell-level says 3,486, pseudobulk says 608. In 65 minutes we will know which to trust

WHERE THIS EPISODE STARTS

Multi-patient tumour data has plenty of moving parts. This episode focuses on three of them: how far to integrate, who is malignant, and what the unit of DE is.

These three come first because they are the ones most likely to send everything downstream off course; settle them and the rest becomes discussable.

Episode goals

1

Combine four patients with Seurat's default integration (Harmony as the comparison), and use positive / negative controls to check under- and over-correction

2

Call malignant cells with inferCNV using immune and oligodendrocyte references, triangulate, write to metadata, check against the authors' labels

3

Run composition analysis, pseudobulk + DESeq2 (paired), GSEA, and use one table to choose the next downstream road

01

Four patients, eight samples

Script 04: load Smart-seq2 counts and GEO metadata

Practice data: Darmanis et al. 2017

tissue

4 GBM patients, each sampled at tumour core and infiltrating periphery

cells

3,589 (author QC passed)

method

Smart-seq2 full-length (no UMI; read counts)

labels

7 types: neoplastic, OPC, astrocyte, oligodendrocyte, neuron, vascular, immune

why

multi-patient+a condition (core vs periphery)+malignant labels to check against

Design: 4 × 2 paired



Two sites from the same patient can be compared as pairs: n=4 pairs

Three things Q2 lacked: multiple patients, a condition (core vs periphery), and author malignant labels to check against

Smart-seq2 vs 10x: what differs, what does not

10x 3' (Q2's GBM 5k)

Smart-seq2 (Q3's GSE84465)

unit	UMI molecules	read counts (PCR duplicates not removed)
total per cell	5k–30k	0.5–several million reads
gene coverage	3' end, 1–3k genes	full length, 4–8k genes
cells / cost	many, cheap	few, expensive, information-rich
QC differences	percent.mt works; catch doublets	percent.mt still works; one cell per well, few doublets
Seurat pipeline	identical	identical (NormalizeData applies)
pseudobulk	sum UMI counts	sum read counts (still integers; DESeq2 works)

The Seurat pipeline is identical; the differences are the counting unit, information per cell, and that doublets are barely a concern

Load counts and metadata

```
# GEO supplementary file (about 20 MB): note it is space-separated, so read.csv returns 0 columns
cnt <- read.table(gzfile("data/GSE84465_GBM_All_data.csv.gz"), header = TRUE,
                 sep = " ", row.names = 1, check.names = FALSE)
bad <- c("no_feature", "ambiguous", "alignment_not_unique",
        "too_low_aQual", "not_aligned") # HTSeq summary rows
cnt <- cnt[!row.names(cnt) %in% bad, ]

# The metadata lives in the series matrix. Do not use getGEO (GSEMatrix = TRUE): an incomplete
# cached download throws "parsing failed--expected only one 'series_data_table_begin'".
# Reading the header lines yourself is the reliable route.
sm <- "data/geo/GSE84465_series_matrix.txt.gz" # already fetched by 00_setup.R
hdr <- grep("^!Sample_characteristics_ch1",
           readLines(gzfile(sm)), wam = FALSE, value = TRUE)
meta <- as.data.frame(do.call(cbind, lapply(hdr, function(x)
  scan(text = sub("^![^  ]+", "", x), what = "", sep = " ",
    quote = "\"", quiet = TRUE))))
```

getGEO's parseGSEMatrix fails easily on this GSE; downloading and reading the header lines ourselves is more reliable | Script 04 §1

Build the object, quick QC

```
# Column order is not guaranteed -> find columns by content, never hard-code ch1.1 / ch1.2
get.val <- function(df, key) {
  hit <- sapply(df, function(v) any(grepl(paste0("^ ", key, ":"), v)))
  sub(paste0("^ ", key, ":"), "", df[names(df)[which(hit)[1]]])
}
meta$patient <- get.val(meta, "patient id"); meta$tissue <- get.val(meta, "tissue")
meta$celltype_author <- get.val(meta, "cell type")
meta$cell <- paste(get.val(meta, "plate id"), get.val(meta, "well"), sep = ".") # e.g. 1001000173.G8
meta <- meta[match(colnames(cnt), meta$cell), ] # align with the counts columns
gbm4 <- CreateSeuratObject(as.matrix(cnt), meta.data = meta, min.cells = 3, min.features = 500)
gbm4[["percent.mt"]] <- PercentageFeatureSet(gbm4, pattern = "^MT-")
table(gbm4$patient, gbm4$tissue)
```

	Periphery	Tumor	
BT_S1	55	433	← all eight present, but a 17-fold spread
BT_S2	445	712	
BT_S4	554	957	
BT_S6	179	204	

Smart-seq2 nFeature runs 4-8k, so the floor is 500 not 200; check the distributions with VlnPlot by patient. The eight samples run from 55 to 957 cells — that imbalance comes back at DE time | Script 04 \$2

02

Integration: how much correction is enough

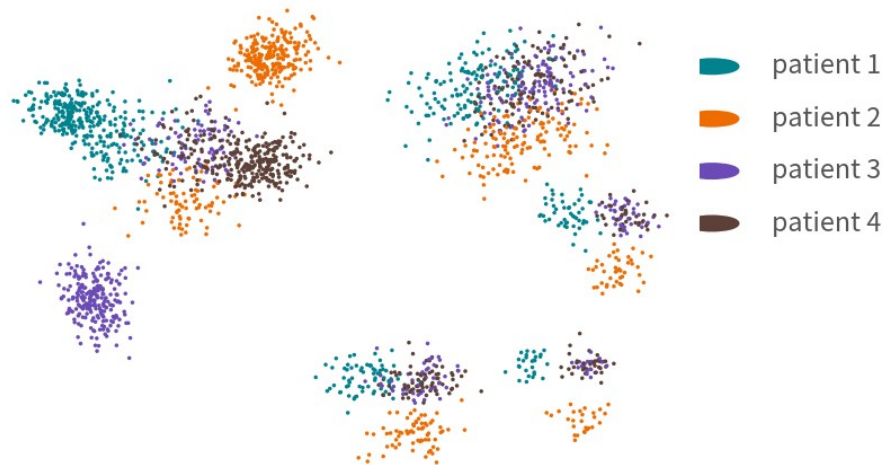
Integration is a dilemma in tumour data; look at the unintegrated plot first

Run the pipeline once without integration and keep the baseline

```
gbm 4 <- NormalizedData(gbm 4) |> FindVariableFeatures() |> ScaleData() |> RunPCA()
gbm 4 <- FindNeighbors(gbm 4, dims = 1:25) |>
  FindClusters(resolution = 0.5, cluster.name = "raw_clusters")
gbm 4 <- RunUMAP(gbm 4, dims = 1:25, reduction.name = "umap.raw")
DimPlot(gbm 4, reduction = "umap.raw",
  group.by = c("patient", "celltype_author"))
saveRDS(gbm 4, "output/rds/04_gbm 4_unintegrated.rds") # the malignant analysis comes back to this one
```

output (schematic)

patient



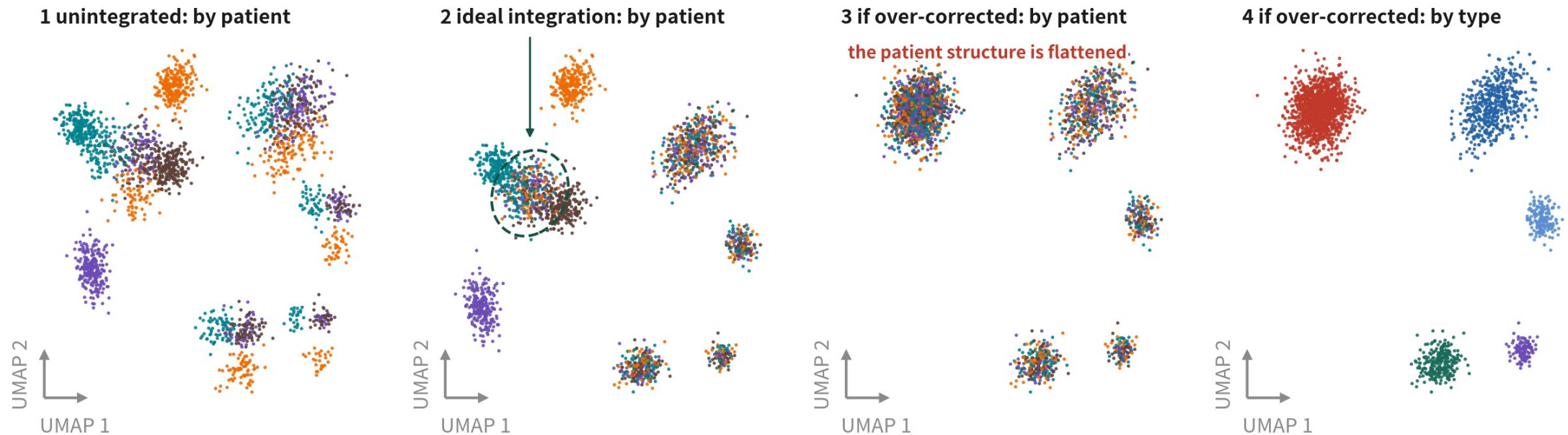
celltype_author



Without a baseline we cannot tell what integration changed. Author labels serve as the known answer here | Script 04 §3 | Deeper: B11

What the unintegrated baseline shows

The integration dilemma: removing the patient effect also removes real malignant biology



The green ellipse marks a state shared across patients: in the ideal result it is expected to mix. Malignant cells are not four unrelated clusters. What must survive is each patient's private component-and if the correction is pushed too far, panel 3 is where even that goes

Practice: use the integrated space only to annotate normal types and align immune cells; analyse malignant cells in the unintegrated space, per patient

schematic / simulated

Panel 2 is the ideal: normal cells blend, the malignant patient structure survives, and the green ellipse - a state shared across patients - is expected to mix. Panel 3 is where too much correction lands you: even each patient's private component has been flattened

The integration map: how hard it corrects, what it needs, the risk for tumours

There is no best method, only a correction strength that matches the question

Method	Idea	Strength	Best for	Risk in tumour data
Seurat CCA / RPCA (default here)	Anchor pairs of cells across samples; built into Seurat v5, one IntegrateLayers call	RPCA weak CCA strong	Samples with high type overlap	CCA over-corrects most; patient-private types get merged into others
Harmony (comparison)	Iteratively pulls batch centroids together in PC space	medium	Many patients / batches fast, light	High theta forces malignant cells together
scVI / scANVI	Variational autoencoder with batch as covariate	tunable	Large atlases, cross-technology (10x + Smart-seq2)	Needs GPU and Python latent space is hard to interpret
No integration, analyse apart	Integrated space for normal cells; malignant cells per patient	—	The mainstream tumour approach	Comparing malignant states across patients needs scores (Neftel), not clusters

References: Stuart T, et al. Cell. 2019;177(7):1888–1902.e21. doi:10.1016/j.cell.2019.05.031 | Korsunsky I, et al. Nat Methods. 2019;16(12):1289–1296. doi:10.1038/s41592-019-0619-0

Integration: Seurat default first, Harmony to compare

```
gbm 4[["RNA"]] <- split(gbm 4[["RNA"]], f = gbm 4$patient) # one layer per patient
gbm 4 <- NormalizeData(gbm 4) |> FindVariableFeatures() |> ScaleData() |> RunPCA()
# (a) Seurat default: CCA anchors (switch to RPCA integration for large data)
gbm 4 <- IntegrateLayers(gbm 4, method = CCAIntegration,
  orig.reduction = "pca", new.reduction = "integrated.cca")
# (b) for comparison: Harmony (the fallback when Seurat integration disappoints)
gbm 4 <- IntegrateLayers(gbm 4, method = HarmonyIntegration,
  orig.reduction = "pca", new.reduction = "harmony")
gbm 4 <- JoinLayers(gbm 4)
for (red in c("integrated.cca", "harmony")) { # same dims and resolution, or they are not comparable
  tag <- sub("integrated.", "", red, fixed = TRUE)
  gbm 4 <- FindNeighbors(gbm 4, reduction = red, dims = 1:25,
    graph.name = paste0(tag, "_snn")) |>
    FindClusters(graph.name = paste0(tag, "_snn"), resolution = 0.5,
    cluster.name = paste0(tag, "_clusters")) |>
    RunUMAP(reduction = red, dims = 1:25,
    reduction.name = paste0("umap.", tag))
}
DimPlot(gbm 4, reduction = "umap.cca", group.by = "patient") |
DimPlot(gbm 4, reduction = "umap.harmony", group.by = "patient")
```

Integration only adds reductions; no expression value changes; DE always returns to RNA counts. Judge Seurat's result first; switch to Harmony only when the positive control fails (immune cells still split by patient) | Script 04 §4 | Deeper: B11

References: Stuart T, et al. Cell. 2019;177(7):1888–1902.e21. doi:10.1016/j.cell.2019.05.031 | Korsunsky I, et al. Nat Methods. 2019;16(12):1289–1296. doi:10.1038/s41592-019-0619-0

Three checks on integration: what should blend, what should survive, was expression altered

	expected (pass)	warning sign (fail)
 positive control: immune	✓ four patients blend → batch corrected	 still clusters by patient→under-corrected
 positive control: oligo, vascular	✓ mix across patients	 still clusters by patient
 negative control: malignant	✓ the patient-specific part survives (shared states are expected to mix)	 even the private part is flattened into one homogeneous blob→over-corrected
 third check: known markers	✓ PTPRC/SOX2/MBP/PECAM1 still mark the same cells before and after	 marker levels flattened or moved→ integration changed more than the coordinates

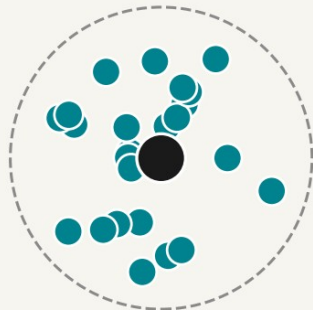
The negative control is not 'did the patients separate' - shared states are expected to mix. Ask whether the private component survives.

The negative control is not 'did the patients separate completely' - shared states are expected to mix; the question is whether the private component survives. Patient entropy here (max 1.386): immune 0.79 to 1.01 passes; malignant 0.17 to 1.09 fails - higher than immune, meaning even the private events were flattened | Script 04 §4

LISI: turning 'did it blend' into a single number

schematic / simulated

1 Look at one cell's neighbours



all neighbours
from one patient
iLISI ≈ 1.0



all four
patients, evenly
iLISI ≈ 4.0

2 The formula

$$\text{LISI} = \frac{1}{\sum_i p_i^2}$$

p_i = the share of neighbours from patient i , distance-weighted (perplexity 30 by default). This is the inverse Simpson index-ecology uses it for effective species count; here it counts effective patients.

Range: 1 (all one patient) to N (N patients evenly mixed). Here $N=4$.

3 Two flavours, opposite directions

iLISI

label=patient. Should rise after integration.

cLISI

label = cell type. Should stay near 1.

Two cautions when reading it:

- 1 A rare type present in only one patient has a naturally low iLISI -that is not under-correction.
- 2 A high iLISI for malignant cells is bad news: the patient-specific part has been erased.

Korsunsky et al. Nat Methods 2019;16:1289-1296 (Harmony, LISI); Tran et al. Genome Biol 2020;21:12

A high iLISI means the batches mixed more; cLISI near 1 means types were not fused. Both only measure mixing — neither alone proves the integration is biologically right: a rare type has a naturally low iLISI, and a high iLISI for malignant cells is bad news

Turn 'did it blend' into numbers: per-cluster patient mix, LSI

```
# 1. how much of each integrated cluster each patient contributes: a normal cluster should hold all four
round(prop.table(table(gbm4$cca_clusters, gbm4$patient), 1), 2)

# 2. LSI: the effective number of patients among a cell's neighbours (1 = all one patient, 4 = evenly mixed)
library(lsi)
for (red in c("pca", "integrated.cca", "harm ony")) {
  emb <- Embeddings(gbm4, red)[, 1:25]
  gbm4[[paste0("lsi_", sub("integrated.", "", red, fixed = TRUE))]] <-
    compute_lsi(emb, gbm4@meta.data, "patient")$patient
}
gbm4@meta.data > group_by(celltype_author) >
  summarise(across(starts_with("lsi_"), ~ median(x)))
```

celltype_author	lsi_pca	lsi_cca	lsi_harm ony	
Immune cell	1.22	1.61	1.87	← positive control: mixes further open
Odontodendrocyte	1.26	2.00	1.91	
OPC	1.28	1.96	1.90	
Neoplastic	1.00	1.64	1.76	← negative control: near 1 = patient signal kept

Normal cells should move toward the patient count (4); malignant should stay near 1. Here both methods blur the malignant cells — CCA blurs them less without losing on the normal ones, so CCA it is | Script 04 §5

Metrics for integration and clustering: what each measures, and when it misleads

No single metric is trustworthy alone; integration needs both 'batches mixed enough' and 'types not fused'

Metric	What it measures	Input	Good direction	When it misleads
iLISI / cLISI	Effective number of batches (i) or types (c) among a cell's neighbours	Embedding + batch (c also needs types)	iLISI high cLISI near 1	A rare type present in only one patient has a low iLISI by construction — not under-correction
kBET	Whether a local neighbourhood's batch mix matches the global mix (rejection rate)	Embedding + batch	Low rejection	When samples genuinely differ in composition — almost always in tumour — rejection is guaranteed
ASW (silhouette)	Batch ASW: how separable batches are; type ASW: how separable cell types are	Embedding + labels	Batch ASW low type ASW high	Purely geometric; elongated or curved groups (differentiating cells) always score poorly
ARI / NMI	Agreement between clustering and a reference labelling	Two labellings	Higher is better	Needs a ground truth; tumour data rarely has one, so author labels stand in as an approximation
Purity	The share of the dominant type inside each cluster	Clusters + types	Higher is better	Rises as clusters get finer — resolution 3.0 makes everything pure and meaningless
Positive / negative controls (this course)	Did what should mix (immune) mix, and did what should stay apart (malignant) stay apart	Embedding + type + patient	Both hold	We must nominate the controls ourselves — but for tumour data this is the one check that actually holds

References: Luecken MD, et al. Nat Methods. 2022;19(1):41–50. doi:10.1038/s41592-021-01336-8 | Büttner M, et al. Nat Methods. 2022;19(1):43–49. doi:10.1038/s41592-021-01336-8 | Tran HTN, et al. Nat Methods. 2022;19(1):43–49. doi:10.1038/s41592-021-01336-8

THE INTEGRATION RULE FOR TUMOUR DATA

Use the integrated space only to annotate normal cells and align immune cells; analyse malignant cells in the unintegrated space, per patient.

Not a compromise — the two populations have different statistical structure. Most tumour papers (Tirosh, Neftel, Puram) do exactly this.

The other road: map cells onto a reference atlas

With a good reference we need not annotate from scratch — normal cells especially

Azimuth



Project our cells onto an annotated reference (PBMC, human brain, lung...) and inherit labels and UMAP coordinates. The easiest route for immune subsets

scArches



Fine-tune the reference model (scVI / scANVI) on our data without re-training the atlas; works across technologies. Python ecosystem

Malignant cells cannot be mapped



No atlas contains this patient's tumour. Mapped malignant cells only get 'the nearest normal type' — like SingleR, a reminder, not an answer

03

Cell identity: calling malignancy with inferCNV

Script 05: calling malignancy — only a dataset that can support it

Three inputs: counts, annotations, gene positions

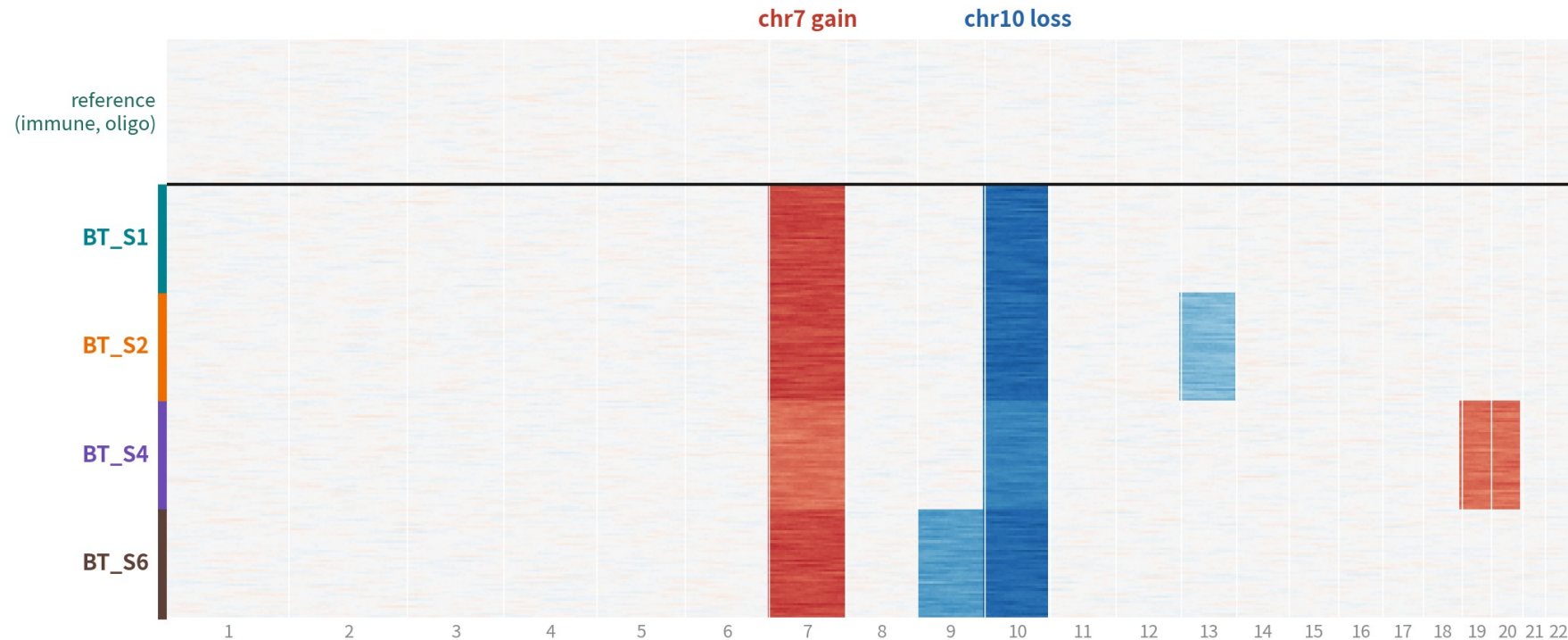
```
library(infercnv)      # BiocManager::install("infercnv")
refs <- c("Immune cell", "Oligodendrocyte")
ann <- data.frame(row.names = colnames(gbm4),
                  group = ifelse(gbm4$celltype_author %in% refs,
                                gbm4$celltype_author,
                                paste0("obs_", gbm4$patient)))
write.table(ann, "data/infercnv_annot.txt", sep = " ",
            colnames = FALSE, quote = FALSE)
# gene position file: the official hg38 gencode build (download link in script 05)
cts <- LayerData(gbm4, layer = "counts")
obj <- CreateInfercnvObject(raw_counts_matrix = cts,
                           annotations_file = "data/infercnv_annot.txt",
                           gene_order_file = "data/hg38_gencode_v27.txt",
                           ref_group_names = refs)
obj <- infercnv::run(obj, cutoff = 1,      # Smart-seq 1; 10x 0.1
                    out_dir = "output/rds/05_infercnv",
                    cluster_by_groups = TRUE, denoise = TRUE, HMM = FALSE)
```

References must be cells known to be normal: immune and oligodendrocytes are safest. Not astrocytes — they may be the malignant ones | Script 05 §1 | Deeper: B16

References: inferCNV of the Trinity CTAT Project. github.com/broadinstitute/inferCNV | Patel AP, et al. Science. 2014;344(6190):1396–1401. doi:10.1126/science.1254257

How to read the inferCNV heatmap

schematic / simulated



Top: reference cells (flat). Bottom: each patient's malignant cells carry their own CNV set—the shared chr7+/chr10– is textbook GBM; the rest is private to each patient

chr7 gain and chr10 loss are textbook GBM, shared by all four; the rest are private to each patient — the genomic reason behind Q1's patient-specific clusters

Key inferCNV parameters: different platforms, different numbers

The commonest error is copying the cutoff from the wrong platform; the next is the wrong reference (Pitfall 1)

Parameter	Meaning	Smart-seq2	10x	Note
cutoff	Drop genes whose mean expression is below this	1	0.1	1 on 10x drops nearly all genes: a blank heatmap
ref_group_names	Who is 'known normal'	Immune + Oligodendrocyte	same	≥ 50–100 cells per group never astrocytes
window_length	Smoothing window along the genome (genes)	101	101	Larger = smoother focal events vanish
HMM / denoise	Segment the signal into discrete CNV states	i6, optional	i6, optional	HMM is slow this course uses score + correlation
cluster_by_groups	How observation cells are grouped in the plot	by patient	by sample	Per patient is what reveals private events

Reference: inferCNV of the Trinity CTAT Project. github.com/broadinstitute/inferCNV

From the heatmap to two per-cell scores

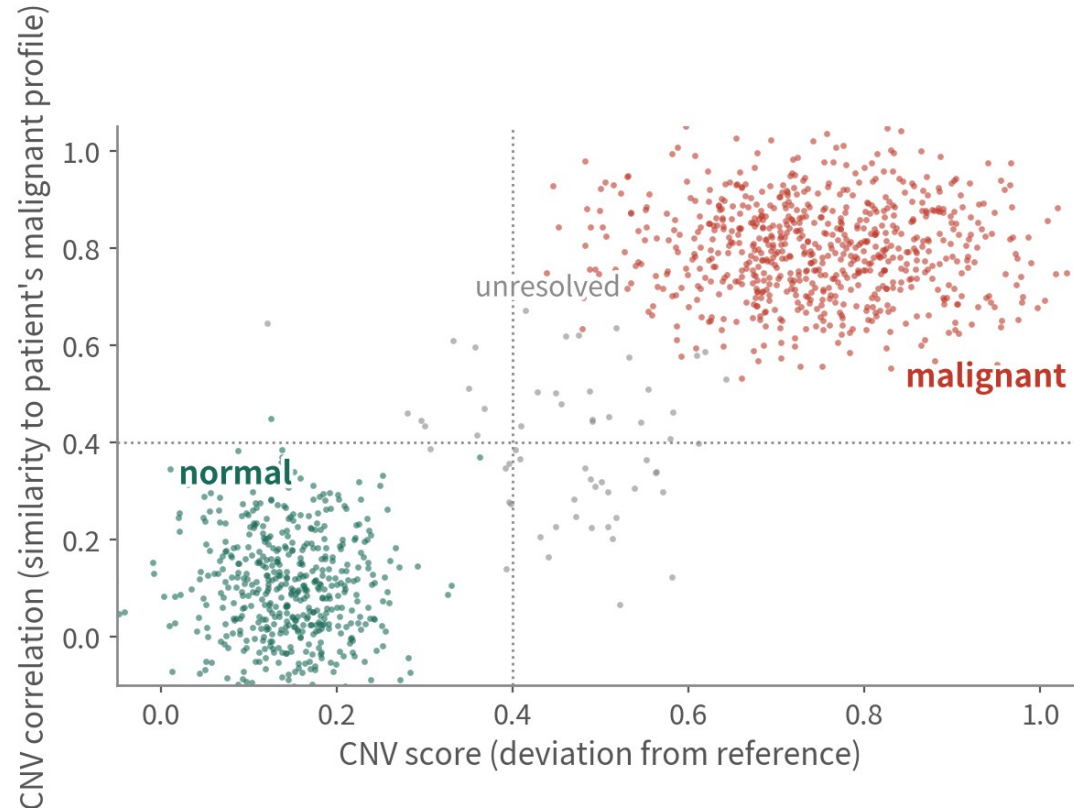
```
# Take the CNV matrix from the object run() returns. Do not read infercnv.observations.txt -
# those text files are only written by plot_cnv(write_expr_matrix = TRUE); run() does not write them.
obj <- readRDS("output/rds/05_infercnv/run.final_infercnv_obj") # only needed after restarting R
cnv.all <- obj@expr.data # genes x cells, smoothed, denoised, centred on the reference

cnv.score <- colMeans((cnv.all - 1)^2) # 1. how far the cells sit from the reference
top <- names(sort(cnv.score, decreasing = TRUE))[1:200]
malp.profile <- rowMeans(cnv.all[, top]) # mean profile of the 200 most malignant-looking cells
cnv.cor <- apply(cnv.all, 2, cor, y = malp.profile) # 2. correlation with that malignant profile

gbm4$cnv.score <- cnv.score[colnames(gbm4)]
gbm4$cnv.cor <- cnv.cor[colnames(gbm4)]
FeatureScatter(gbm4, "cnv.score", "cnv.cor", group.by = "celltype_author")
```

This is Tirosh 2016's approach: score plus correlation, plotted to see whether two clear groups emerge | Script 05 §2

Two numbers per cell, four quadrants



schematic / simulated

Triangulate: all three independent lines of evidence before we call it

① **CNV**

high score+high correlation with the patient's malignant profile

② **clustering**

mostly cluster per patient; normal cells mix across patients

③ **markers**

glial lineage (SOX2, OLIG2), not immune/vascular

into metadata

three labels; do not force the unsure ones

Two-axis call: Tirosh et al. Science 2016;352:189-196. Triangulation: Neftel et al. Cell 2019;178:835-849

Top-right malignant, bottom-left normal, the middle unresolved. Do not force the unsure ones — reviewers prefer an honest grey zone

Reference: Tirosh I, et al. Science. 2016;352(6282):189-196. doi:10.1126/science.aad0501

Triangulation: write to metadata, compare with the authors' labels

```
ref.cell<-gbm 4$celltype_author%in% refs      # thresholds grow out of the reference cells
s.hi<-quantile(gbm 4$cnv.score[ref.cell],0.99) # ceiling of a normal CNV score
c.hi<-quantile(gbm 4$cnv.cor[ref.cell], 0.99) # ceiling of a normal correlation
c.lo<-quantile(gbm 4$cnv.cor[ref.cell], 0.90) # below this line = clearly normal
gbm 4$malignant<-with(gbm 4@meta.data,ifelse(
  cnv.score > s.hi& cnv.cor> c.hi,"malignant",
  ifelse(cnv.score <= s.hi& cnv.cor< c.lo,"normal","unresolved")))
# Third evidence: lineage. In immune /vascular/oligodendrocyte /neuron go back to unresolved
gbm 4$malignant[gbm 4$malignant== "malignant" & gbm 4$celltype_author%in%
  c("Immune cell", "Vascular", "Oligodendrocyte", "Neuron")]<- "unresolved"
table(gbm 4$malignant,gbm 4$celltype_author)
```

	Neoplastic	Astrocyte	0 PC	0 oligodendrocyte	Immune cell	Vascular	Neuron
malignant	high	few	few	0	0	0	0
normal	few	most	most	all	all	all	most
unresolved	...						

Check against the authors' Neoplastic label: our run gives 87.6% agreement, 99.4% precision, 59.1% recall. The disagreements are worth a look — often periphery cells | Script 05 §3

Evidence of malignancy beyond CNV

CNV-quiet tumours (paediatric brain tumours, some blood cancers, near-diploid carcinomas) need different independent lines of evidence

Mutations and alleles



Smart-seq2 reads coding SNVs (IDH1 R132H, TP53); 10x catches them only near the 3' end. HoneyBADGER and Numbat add allelic imbalance to CNV

A second CNV tool



copyKAT needs no reference; SCEVAN is automatic. Call malignant only where all agree; leave the rest unresolved

Expression programmes and pathology



High Neftel state scores, high proliferation, unlike any normal type; plus pathology (e.g. IDH1 R132H immunostain). Supporting, never sole evidence

04

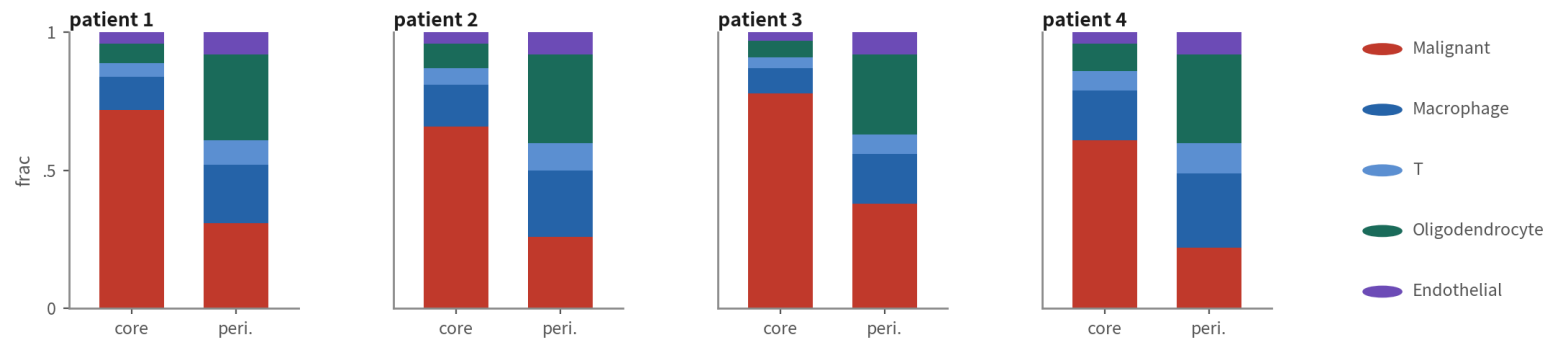
Composition: what differs between core and periphery

Script 06a §1: proportions are compositional; the unit is the patient

Type proportions per patient and site

```
gbm 4$type <- ifelse(gbm 4$malignant == "malignant", "Malignant", gbm 4$celltype_author)
prop <- gbm 4@meta.data %>% dplyr::count(patient, tissue, type) %>%
  group_by(patient, tissue) %>% mutate(frac = n / sum(n))
ggplot(prop, aes(tissue, frac, fill = type)) + geom_col() +
  facet_wrap(~ patient, nrow = 1) + theme_classic()
library(speckle); library(linma) # unit = the sample (8 of them), paired design
p8 <- getTransformedProps(gbm 4$type, paste(gbm 4$patient, gbm 4$tissue, sep = "_"),
  transform = "logit")$TransformedProps
pat <- sub("_[^_]*$", "", colnames(p8)) # patient IDs contain "_": split on the last one
tis <- factor(sub(".*_", "", colnames(p8)), c("Periphery", "Tumor"))
topTable(eBayes(lmFit(p8, model.matrix(~ pat + tis))), coef = "tisTumor", n = Inf)
```

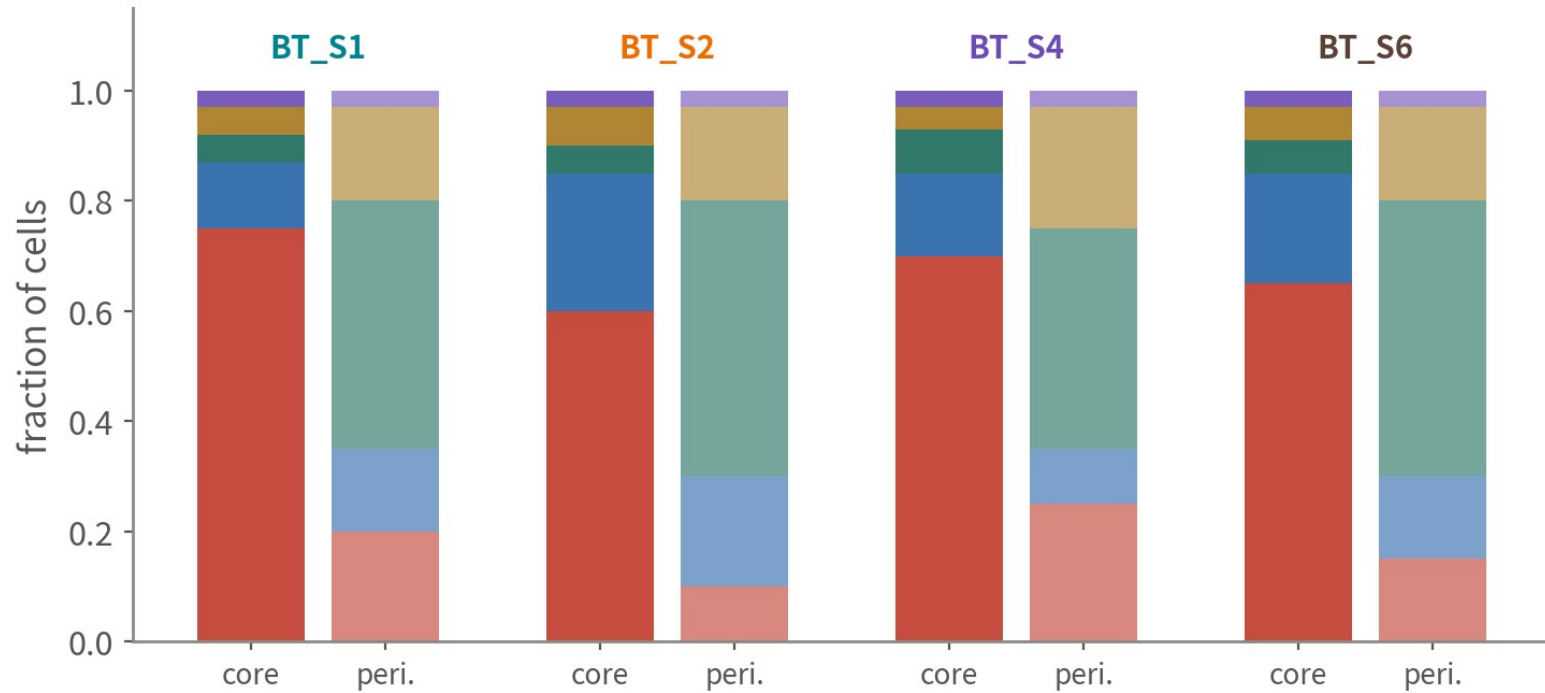
output (schematic)



'Fewer malignant cells at the periphery' is tested with propeller / scCODA on n = 8 samples; a t-test on proportions is wrong | Script 06a §1 | Deeper: B12

Composition: one pair per patient

schematic / simulated



- malignant
- immune
- oligo
- OPC / astro / neuron
- vascular

Proportions are compositional

Fewer malignant cells at the periphery may just mean more normal brain. To test proportions use propeller/scCODA, with patients as units

It counts only if all four patients agree. Proportions are closed: one type falling forces the others up

Three things about composition: closure, pairing, the unit

Closure



Proportions sum to 1:
fewer malignant cells
'automatically' means
more immune cells. Test
on the logit or CLR scale
(propeller, scCODA),
never a t-test on raw
proportions

Pairing



Core and periphery come
from the same patient;
a paired design removes
between-patient
variance and boosts
power. In propeller, a
~ patient + tissue
design matrix does it

The unit is the sample



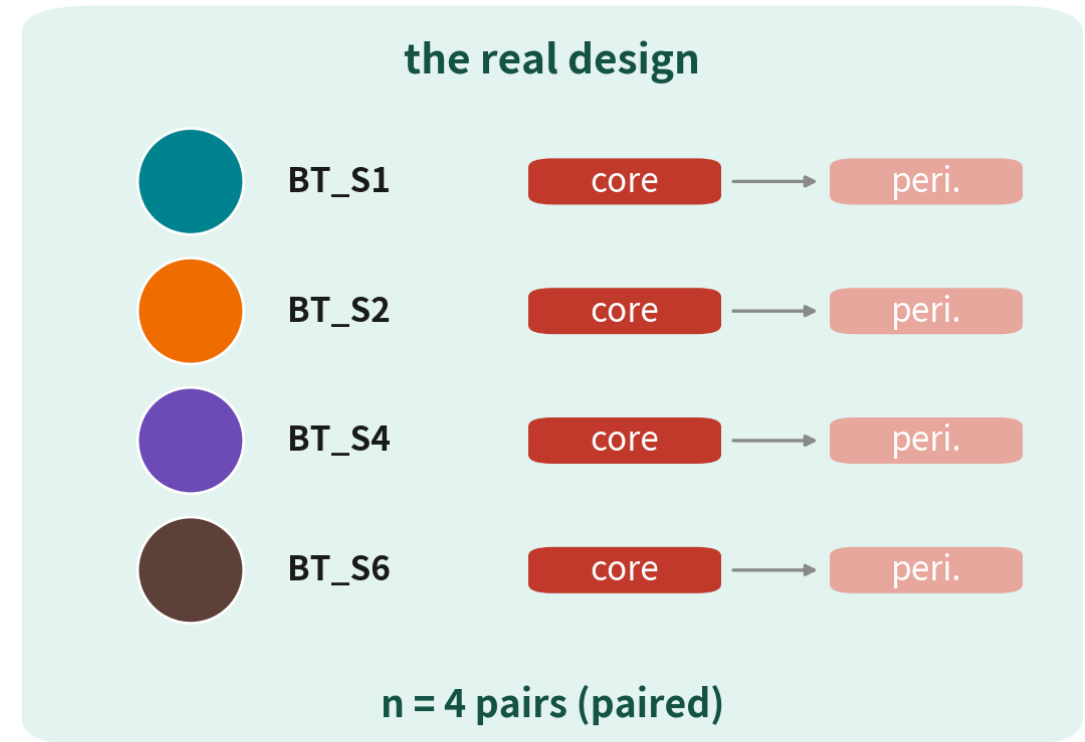
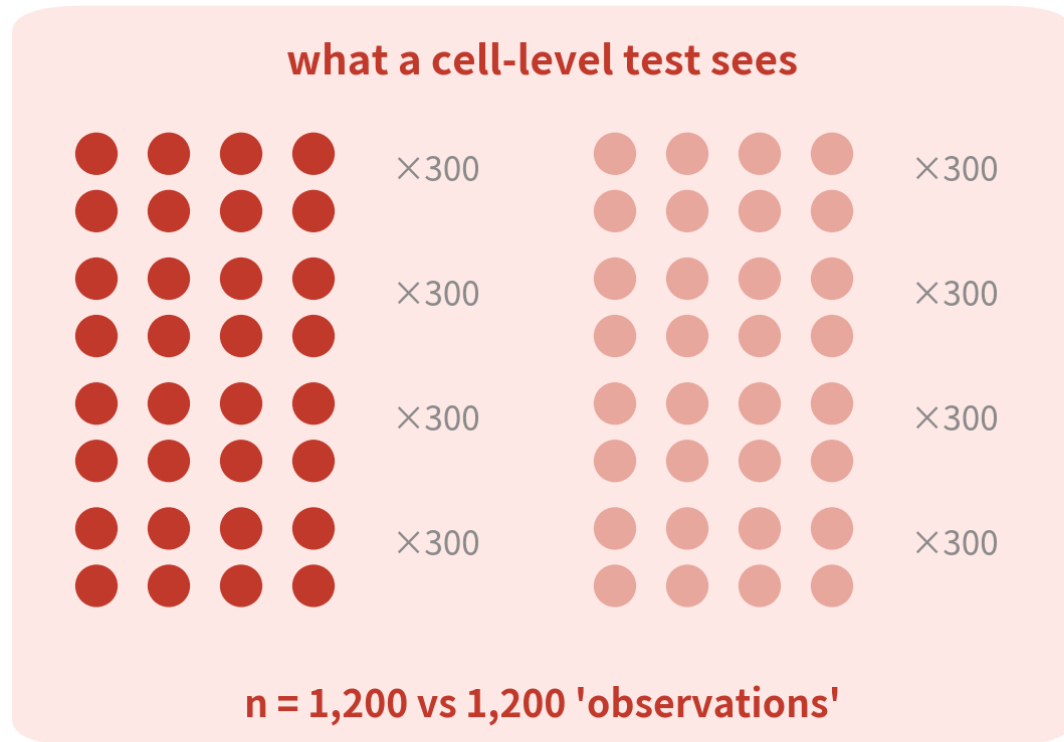
$n = 8$ samples (4×2),
not 3,589 cells. A
chi-square at the cell
level gives absurdly
small p values — the same
mistake as Pitfall 1

05

DE: each cell type, core vs periphery

Scripts 06a (the pseudobulk mainline) and 06b (the cell-level fallback for too few samples)

Why a cell-level test hands back two thousand genes

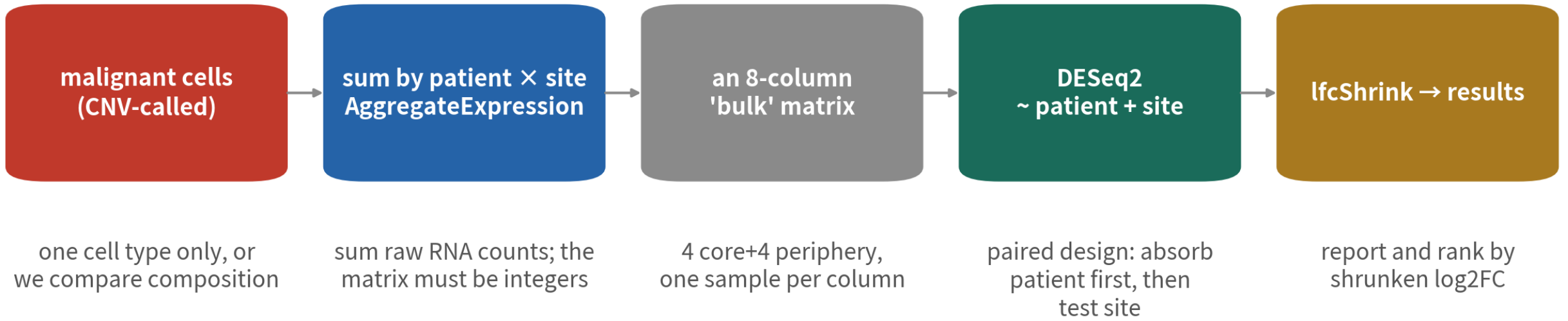


Cells from one patient share genome and CNV—not independent. Treating cells as observations=pseudoreplication; p values collapse

Malignant cells from one patient share one CNV set and are highly correlated. The real design is 4 pairs, not 2,400 observations

Pseudobulk in five steps

Pseudobulk: turn single-cell data back into one row per sample, then use mature bulk statistics



One-line check: `sum(mat != round(mat))` must be 0; otherwise we summed the data / scale.data / integrated layer

One cell type at a time — otherwise we are comparing composition (more normal brain at the periphery), not that type's change

Each cell type separately: why, and the thresholds

Mixed together = comparing composition



Core vs periphery differs in malignant cells, macrophages and oligodendrocytes in different ways. DE on all cells puts MBP and PLP1 on top — that is just more oligodendrocytes at the periphery

Two thresholds



MIN_CELLS = 20: a patient × site needs ≥ 20 cells to count as a pseudobulk sample; drop the cell otherwise. MIN_PAIRS = 2: at least two patients with both sites, or the paired design cannot exist

Only one type clears the gates here



Across all four patients the periphery holds 31 malignant cells in total (17 in the best patient); nobody reaches MIN_CELLS = 20, so core vs periphery in malignant cells cannot be done here. The type that does run is Immune cell: 4 patient pairs, 608 significant genes

Wrap pseudobulk + paired DESeq2 in a function; call it per type

```
pb_de <- function(obj, type, m.in.cells = 20, m.in.pairs = 2) {  
  sub <- subset(obj, cells = colnames(obj)[obj$type == type])  
  n <- table(sub$patient, sub$tissue)  
  ok <- names(which(rowSums(n) >= m.in.cells) == 2) # patients with enough cells at both sites  
  if (length(ok) < m.in.pairs) return(NULL)  
  sub <- subset(sub, cells = colnames(sub)[sub$patient %in% ok])  
  pb <- as.matrix(AggregateExpression(sub, assays = "RNA",  
    group.by = c("patient", "tissue"))$RNA)  
  stopifnot(sum(pb != round(pb)) == 0) # one line that catches the trap: must be integers  
  coldata <- data.frame(patient = sub("_[^_]*$", "", colnames(pb)),  
    tissue = factor(sub(".*_", "", colnames(pb)),  
      levels = c("Periphery", "Tumor")))  
  dds <- DESeqDataSetFromMatrix(pb[rowSums(pb) >= 10, ], coldata,  
    design = ~ patient + tissue)  
  raw <- results(dds, name = "tissue_Tumor_vs_Periphery") # stat -> GSEA and the volcano  
  shr <- lfcShrink(dds, coef = "tissue_Tumor_vs_Periphery", type = "ashr")  
  list(res = data.frame(gene = rownames(raw), log2FC = raw$log2FoldChange,  
    log2FC_shrunk = shr$log2FoldChange, stat = raw$stat,  
    padj = raw$padj), pb = pb, coldata = coldata)  
}  
de <- lapply(types, function(ty) pb_de(gbm4, ty)); names(de) <- types
```

~ patient + tissue: the paired design absorbs patient differences first. Patient IDs contain underscores — split the column name at its last segment | Script 06a §2 | Deeper: B12

Reference: Love MI, Huber W, Anders S. Genome Biol. 2014;15(12):550. doi:10.1186/s13059-014-0550-8

One DE result, three log2FCs: which goes where

Shrinkage pulls the exaggerated FC of low-count, noisy genes toward 0; how hard it pulls differs a lot by method

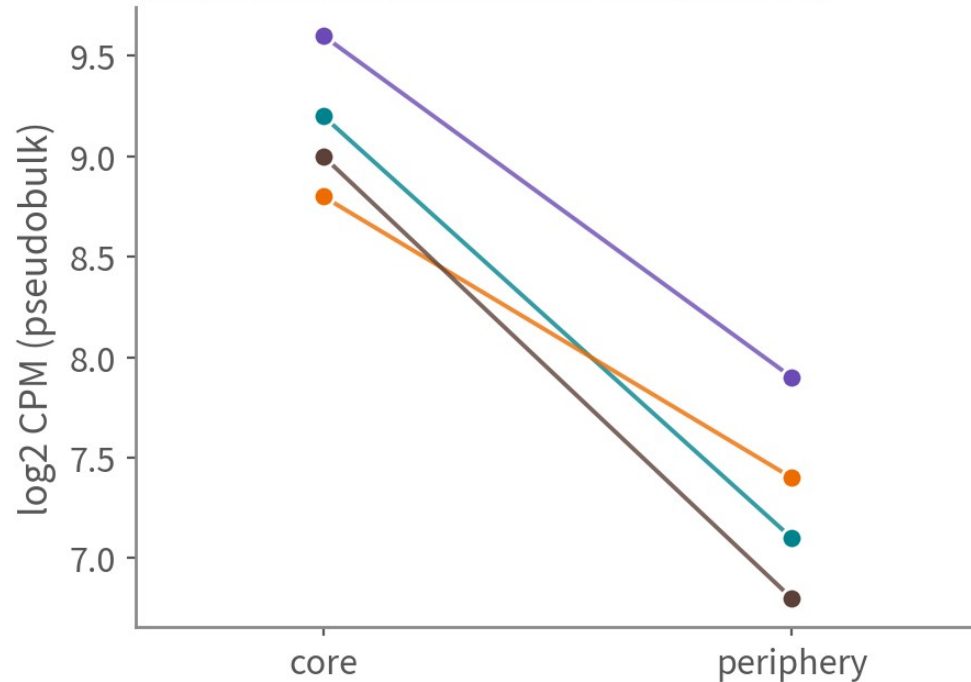
Column	Where it comes from	Behaviour	Use it for
raw log2FC	log2FoldChange from results(dds)	Low-count genes: huge FC, huge SE	The volcano x-axis (keeps the shape honest)
stat (Wald)	log2FC ÷ SE	Carries effect size and uncertainty; few ties	The GSEA ranking statistic
log2FC_shrunk (ashr)	lfcShrink(type = "ashr")	Stable at small n; keeps large effects, damps noise	Reported effect sizes; picking genes to validate
apegglm (unsuited here)	lfcShrink(type = "apegglm")	Here at n = 8 shrinkage is severe: nearly every gene lands on one value → a bar-shaped volcano and all-tied GSEA ranks (how strong depends on the data, not a universal rule)	Only with more samples (≥ 3 vs 3, ample counts)

Reference: Love MI, Huber W, Anders S. Genome Biol. 2014;15(12):550. doi:10.1186/s13059-014-0550-8

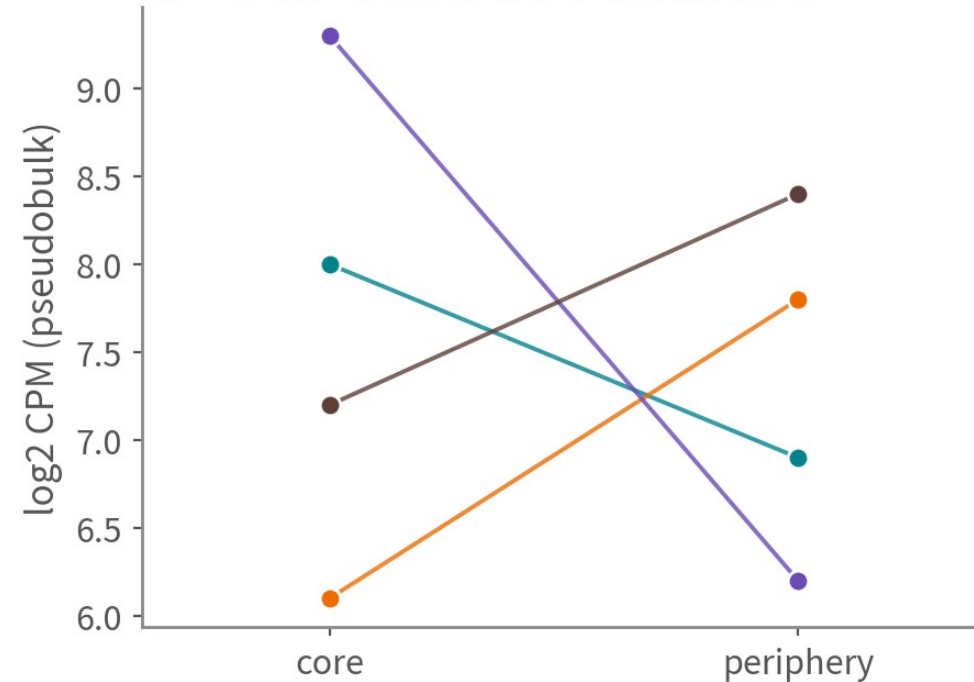
When reporting, show it holds per patient

schematic / simulated

✓ SLC2A1: consistent in all four patients



✗ GENE_X: inconsistent across patients



The left plot is more convincing than any volcano; the right one is still 'hugely significant' in a cell-level test

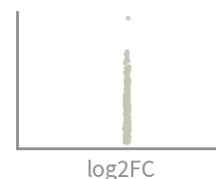
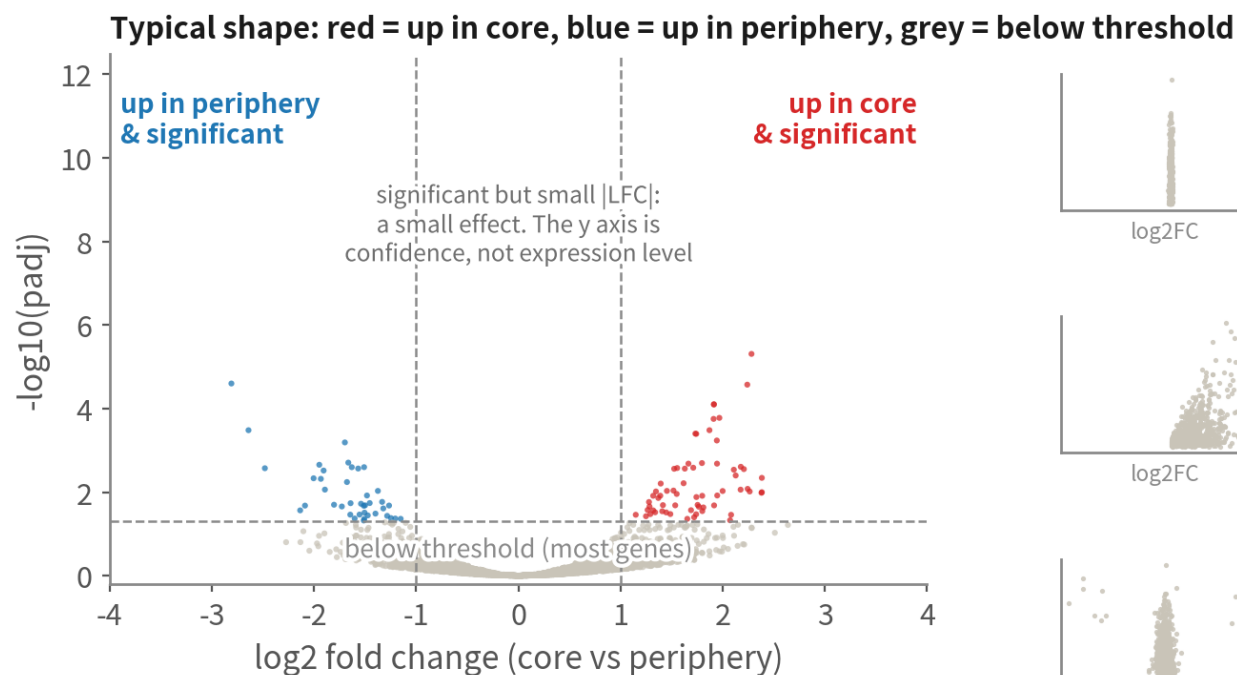
One line per patient (schematic; SLC2A1 is a real hit from this course's run, log2FC 5.7, padj 3.6e-12).

The left is more convincing than any volcano; the right is still 'hugely significant' at cell level

Reading a volcano: shape first, then look for the cause

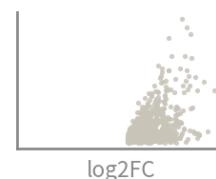
Three shape clues, and what to go back and check

schematic / simulated



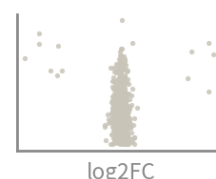
Clue 1 | points collapse into a vertical bar

First ask which LFC is plotted. If it is the shrunken one, many effect sizes may have been pulled toward 0 (common with apegglm at small n)-compare against the unshrunk LFC. If it is already the unshrunk LFC (what this course plots), shrinkage cannot be the cause -check how the LFC was computed, rounding, and the plotting code.



Clue 2 | almost everything on one side

Not necessarily an error. It can be a real directional change, or come from sample imbalance, library-size differences, low counts and excess zeros, an outlier sample, or a shift in cell composition.->Go back to sample-level counts, per-sample cell numbers and individual samples.



Clue 3 | a few points fly out past $|\log_2\text{FC}|$ 10-20

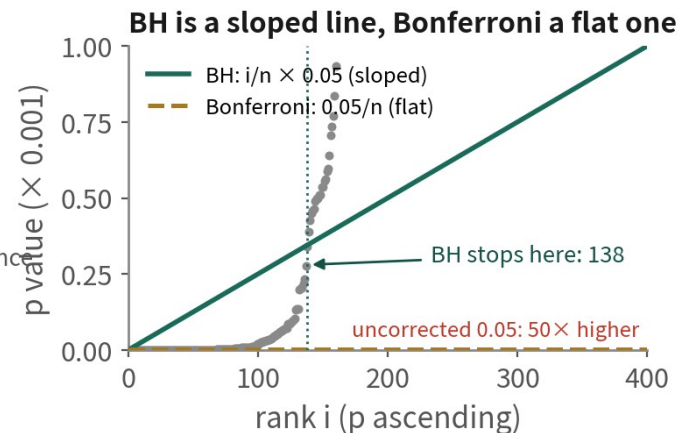
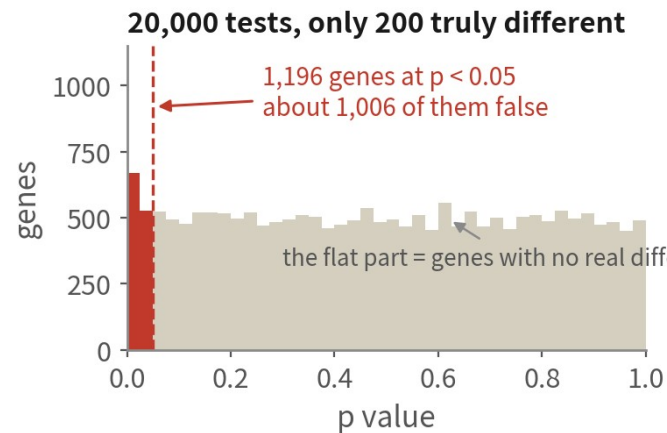
Usually one group's expression is near zero. Do not read it as a millionfold increase.->Check the raw and pseudobulk counts, the sample size and the cell composition; make sure it is not carried by a handful of cells or a single sample.

A volcano is a QC clue, not a diagnosis: asymmetry is not an error in itself, since the biology may genuinely go one way-what should stop you is an extreme or implausible shape. x=unshrunk $\log_2\text{FC}$, y= $-\log_{10}(\text{padj})$, one plot per cell type (script 06 § 3).

Shape is a clue, not a diagnosis: a bar means first check whether the LFC is shrunken; one side may be real direction or sample imbalance or an outlier; extreme $|\log_2\text{FC}|$ usually means one group is near zero - go back to the counts | Script 06a §3

Twenty thousand tests: multiple comparisons and FDR

schematic / simulated



One set of p values, three treatments

No correction

about 1,006 are false positives

1,196

Bonferroni

almost never wrong, but only these of the 200 real ones

78

BH (padj)

about 5 false - close to the 5% promised

138

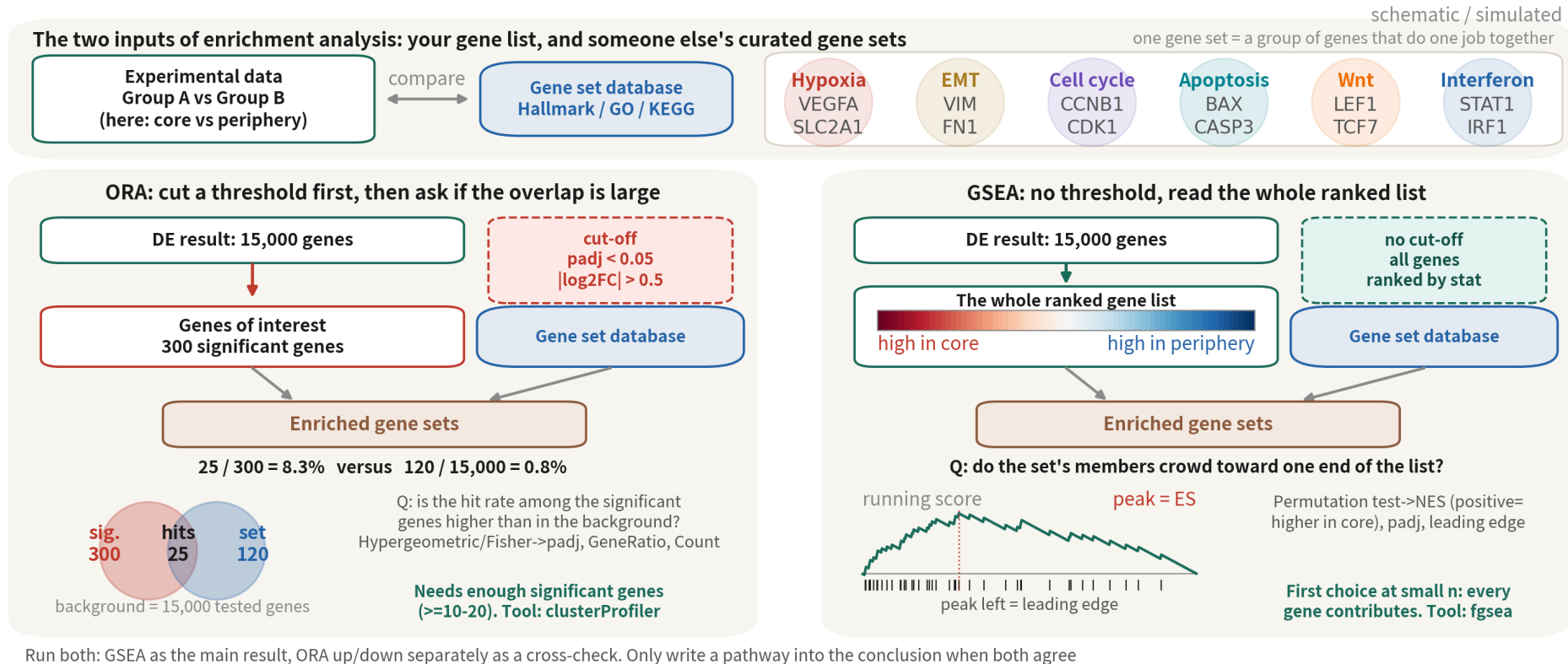
What padj = 0.05 actually means

'Among the genes I am calling significant, about 5% are false'-a rate over the whole set, not 'this one gene has a 5% chance of being false'.

So report padj in the DE table and p.adjust in enrichment, and correct within the set of tests actually run: one DESeq2 run per cell type corrects within that type. Do not pool six types' p values and correct once.

BH is not cosmetic: it changes the question from 'is there any false positive at all' to 'what fraction of this list is false' - which is the right question for genome-scale data | Script 06a §2

Same DE result, two enrichment questions: ORA counts, GSEA ranks



GSEA uses every gene with no threshold — first choice at small n; ORA needs enough significant genes and the right background. Run both; a pathway goes into the conclusion only when both agree

Three gene-set databases: Hallmark, GO, KEGG

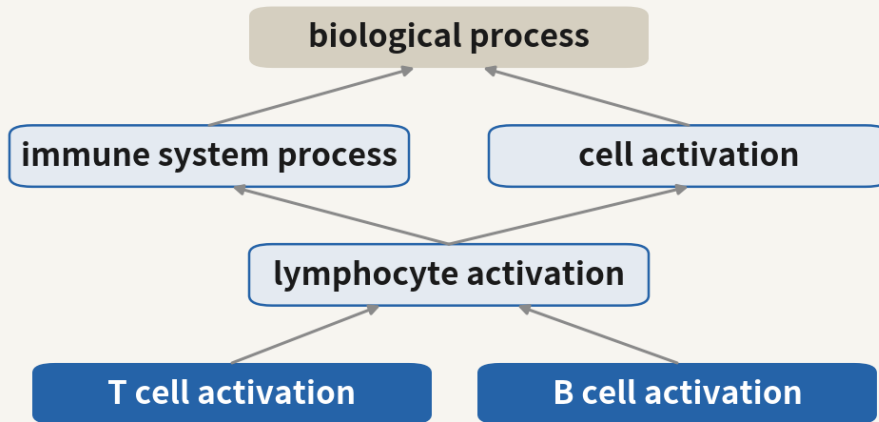
All from msigdb and run with the same fgsea; ORA via clusterProfiler enrichGO / enrichKEGG

Database	Content and size	Strength	Caveat
Hallmark (H)	50 curated, minimally overlapping programmes: hypoxia, EMT, interferon, G2M...	Look here first: fits on one page, easiest to read, almost no redundancy	Only 50 sets; fine detail (which signal, which metabolic branch) is out of reach
GO BP (C5)	About 7,500 BP terms hierarchical, heavily overlapping	The finest grain; the default language of ORA; runs offline (org.Hs.eg.db)	One biology shows up as ten terms — simplify() first; terms with 3–5 hits stay out of the conclusion however significant
KEGG (C2 CP)	About 190 metabolic and signalling pathways, with pathway maps	Most intuitive for metabolism (glycolysis, OXPHOS); pathview draws on the map	enrichKEGG needs the internet; many are disease pathways ('Alzheimer disease') that are not mechanisms; the msigdb copy is the legacy version

Reference: Liberzon A, et al. Cell Syst. 2015;1(6):417–425. doi:10.1016/j.cels.2015.12.004

Why GO returns twenty terms all saying the same thing

GO is a directed acyclic graph, not a list



One CD3E is counted into every level above

Arrows are is_a/part_of. ORA runs one Fisher test per term, and parent and child share the same genes-so significant terms come out in clusters.

A GO result before de-duplication

- T cell activation
- lymphocyte activation
- leukocyte activation
- cell activation
- T cell differentiation
- cytokine production
- regulation of cytokine production
- response to hypoxia

these five green terms say one thing (parent and child of one family)
eight terms, three colour groups:
immune activation, cytokine production, hypoxia

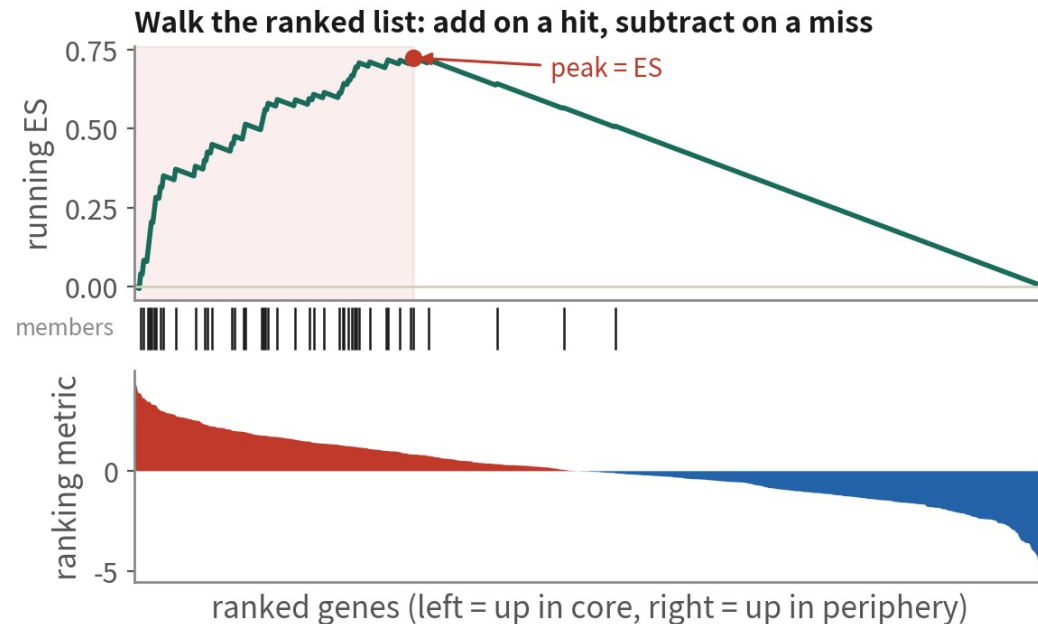
Three ways to handle it

- 1 clusterProfiler simplify(cutoff=0.7): merges semantically similar terms into one representative
- 2 Metascape: clusters terms first and reports one headline term per cluster
- 3 Report BP/CC/MF separately; keep the most specific term of each family, the rest go to the supplement

GO is a hierarchy, not a list: parent and child share the same genes and ORA tests every term separately, so a whole family lights up. De-duplicate before reporting | Script 06a §4

Inside GSEA: the running score, the peak and the leading edge

schematic / simulated



The members inside the red band are the leading edge: the core genes carrying this pathway-that is the list to put in the paper

Four things to know before reading GSEA

- 1 The ranking metric decides everything. Here: $\text{sign}(\log_2\text{FC}) \times -\log_{10}(p)$; bulk GSEA often uses signal-to-noise. Change the metric and the ranking changes-so it goes in Methods.
- 2 The p value comes from a permutation test. With few samples (pseudobulk of four patients) permute the gene set, not the phenotype-phenotype permutation needs many samples.
- 3 NES divides ES by the mean of the random distribution so gene sets of different sizes compare. The sign is direction only.
- 4 The leading edge is the checkable part. 'Hypoxia up' carries no information; 'Hypoxia up, leading edge VEGFA, CA9, SLC2A1' does.

GSEA sets no threshold; it asks whether a gene set clusters at one end of the ranking. What goes in the paper is not the pathway name but the leading-edge gene list | Script 06a §4

GSEA: three databases, every type, one loop

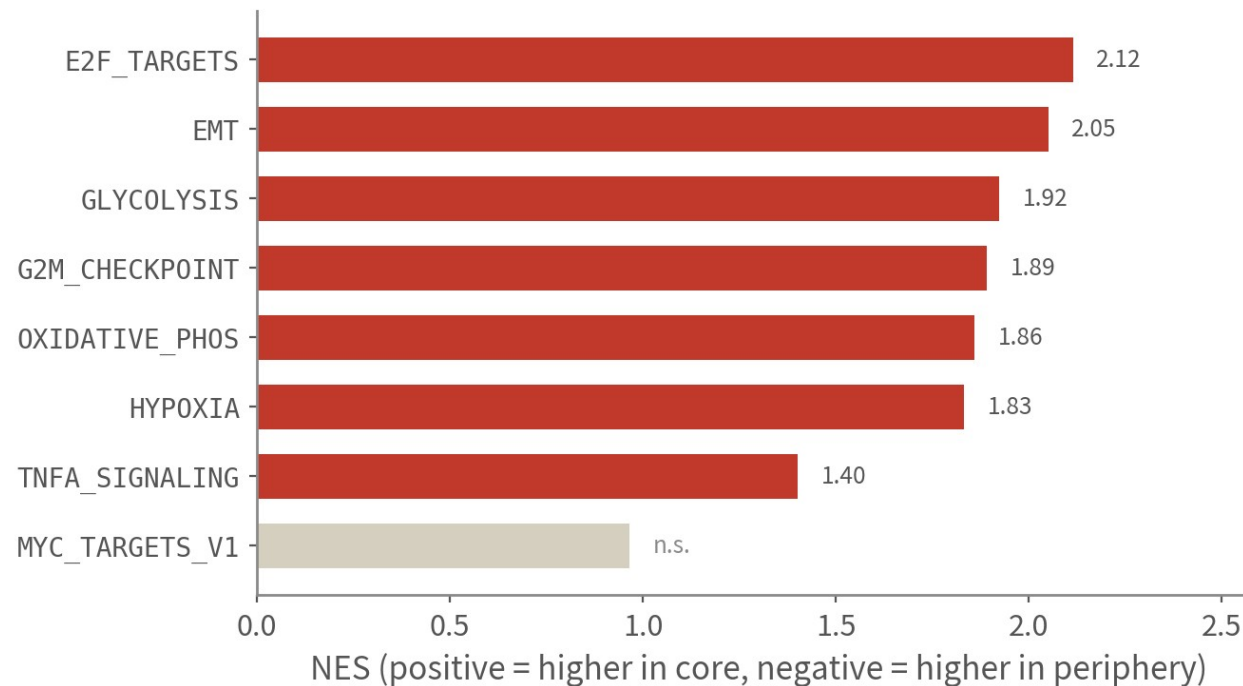
```
library(fgsea); library(msigdbr)
m_sig <- function(coll, sub = NULL) {
  m <- msigdbr(species = "Homo sapiens", collection = coll, subcollection = sub)
  split(m$gene_symbol, m$gs_name)
}
gene_sets <- list(Hallmark = m_sig("H"), GO_BP = m_sig("C5", "GO:BP"),
                 KEGG = m_sig("C2", "CP:KEGG_LEGACY"))
rank_stat <- function(res) { # rank by the Wald stat, not the shrunken LFC
  r <- setNames(res$stat, res$gene); r <- r[!is.na(r)]
  sort(r + mom(length(r), sd = 1e-6), decreasing = TRUE) # break ties
}
run_gsea <- function(d, sets) rbindlist(lapply(names(sets), function(nm) {
  g <- fgsea(sets[[nm]], rank_stat(d$res), minSize = 15, maxSize = 500, nproc = 1)
  g$collection <- nm; g$type <- d$type; g}))
gsea_all <- rbindlist(lapply(de, run_gsea, sets = gene_sets))
gsea_all[adj < 0.05][order(-abs(NES))][, SD[1:min(5, N)], by = .(type, collection)]
```

eps = 0 demands infinitely precise p values and runs for hours; the default 1e-10 is plenty. The background is automatically the tested genes | Script 06a §4 | Deeper: B13

Reference: Korotkevich G, Sukhov V, Sergushichev A. bioRxiv. doi:10.1101/060012

GSEA: turn a hundred-odd genes into one sentence

schematic / simulated



How to read

All seven point to the core here: hypoxia, glycolysis and EMT fit a necrotic centre, and proliferation (E2F, G2M) plus OXPHOS are higher in the core too. MYC is not significant.

Seven in one direction deserves one question first: is this a depth or cell-number difference? Once that is ruled out, read the leading-edge genes

Rank by the DESeq2 Wald stat (not the shrunken log2FC: massive ties) with no significance cut; the background is automatically the tested genes

All seven Hallmark sets are higher in the core here (hypoxia, glycolysis, EMT, proliferation, OXPHOS); MYC is not significant. Agreement this uniform means ruling out a depth difference first, then reading the leading-edge genes

ORA: GO and KEGG in clusterProfiler, up and down separately

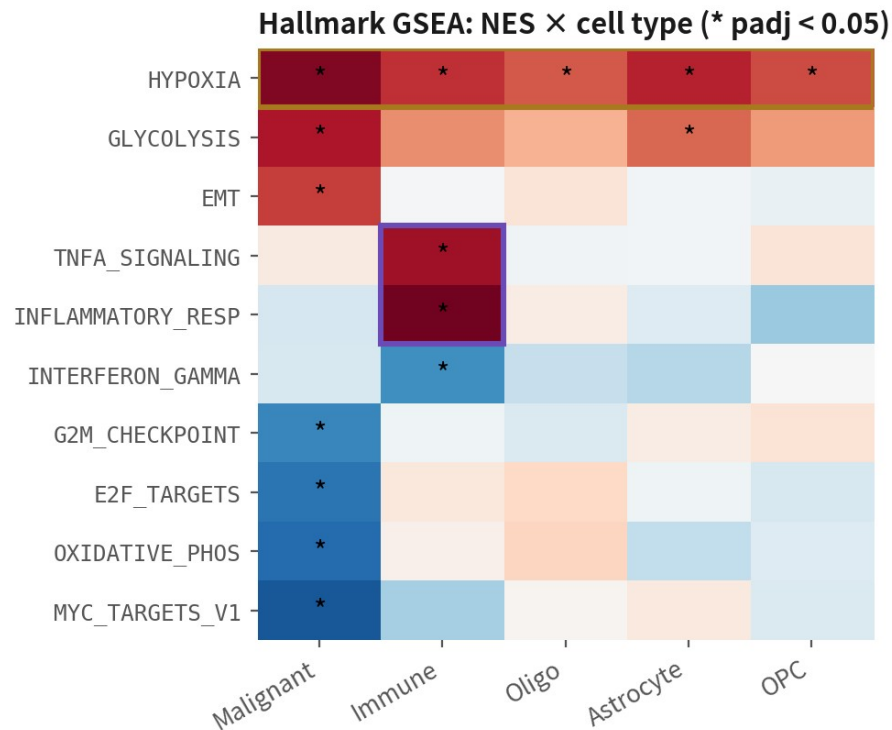
```
library(clusterProfiler); library(org.Hs.eg.db); library(enrichplot)
run_ora <- function(d, direction = "up", fc = 0.5, p = 0.05) {
  r <- d$res[!is.na(d$res$padj), ]
  sig <- r$gene[r$padj < p & (if (direction == "up") r$log2FC > fc else r$log2FC < -fc)]
  if (length(sig) < 10) return(NULL) # too few significant genes: use GSEA instead
  go <- enrichGO(sig, OrgDb = org.Hs.eg.db, keyType = "SYMBOL", ont = "BP",
    universe = d$res$gene, # * background = the genes tested, not the whole genome
    minGSSize = 15, maxGSSize = 500) |>
    clusterProfiler::simplify(cutoff = 0.7) # igraph has simplify() too, so write the full name
  ids <- bitr(sig, "SYMBOL", "ENTREZID", org.Hs.eg.db)
  uni <- bitr(d$res$gene, "SYMBOL", "ENTREZID", org.Hs.eg.db)
  kegg <- enrichKEGG(ids$ENTREZID, "hsa", universe = uni$ENTREZID, minGSSize = 15) |>
    setReadable(org.Hs.eg.db, keyType = "ENTREZID") # needs an internet connection
  list(go = go, kegg = kegg) }
ora <- run_ora(de["Immune cell"], "down") # the only type that clears the bar
dotplot(ora$go, showCategory = 12); dotplot(ora$kegg, showCategory = 12)
cnetplot(ora$go, showCategory = 5) # term-gene network: which genes carry which term
```

Leave universe empty and clusterProfiler uses the whole genome as background — twice the terms, all spurious. Up and down separately, because 'hypoxia up' and 'proliferation down' cancel when mixed. We demo the down side: this dataset returns no terms on the up side | Script 06a §4

Reference: Wu T, et al. Innovation (Camb). 2021;2(3):100141. doi:10.1016/j.xinn.2021.100141

Reading enrichment: the NES heatmap and the dotplot

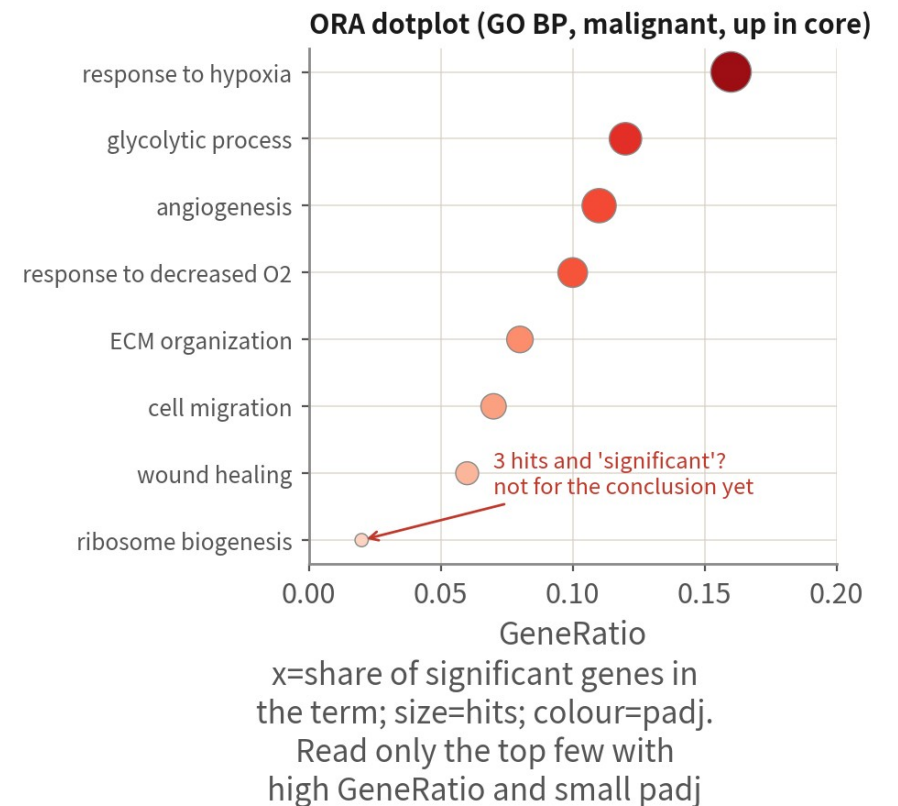
schematic / simulated



whole row red: a microenvironment programme (hypoxia)—every cell type feels it

one column only: a cell-intrinsic programme (inflammation in immune, proliferation in malignant)

Tells us at a glance whose pathway it is—a question a mixed-cells DE cannot even ask



A whole red row = a microenvironment programme every type feels; one column = a cell-intrinsic programme.

In the dotplot read only the top few with high GeneRatio and small padj | Script 06a §4, plots A–C

The five most common enrichment mistakes

Wrong background



The universe must be the tested genes, not the genome. Single cell detects ~12k genes; a 20k background inflates every term

Direction flipped or mixed



NES sign follows the tissue level order. Unsplit ORA lets opposite programmes cancel — check direction first

Redundancy is not evidence



GO's three 'response to ... oxygen' terms are one thing. Count after simplify(); report one term, not three

Concluding from too few hits



Terms with 3–5 hits stay out of the conclusion. A leading edge needs a dozen genes we can explain

Whose pathway is it?



Hypoxia rises in malignant cells and macrophages alike — that is the microenvironment, not a malignant trait

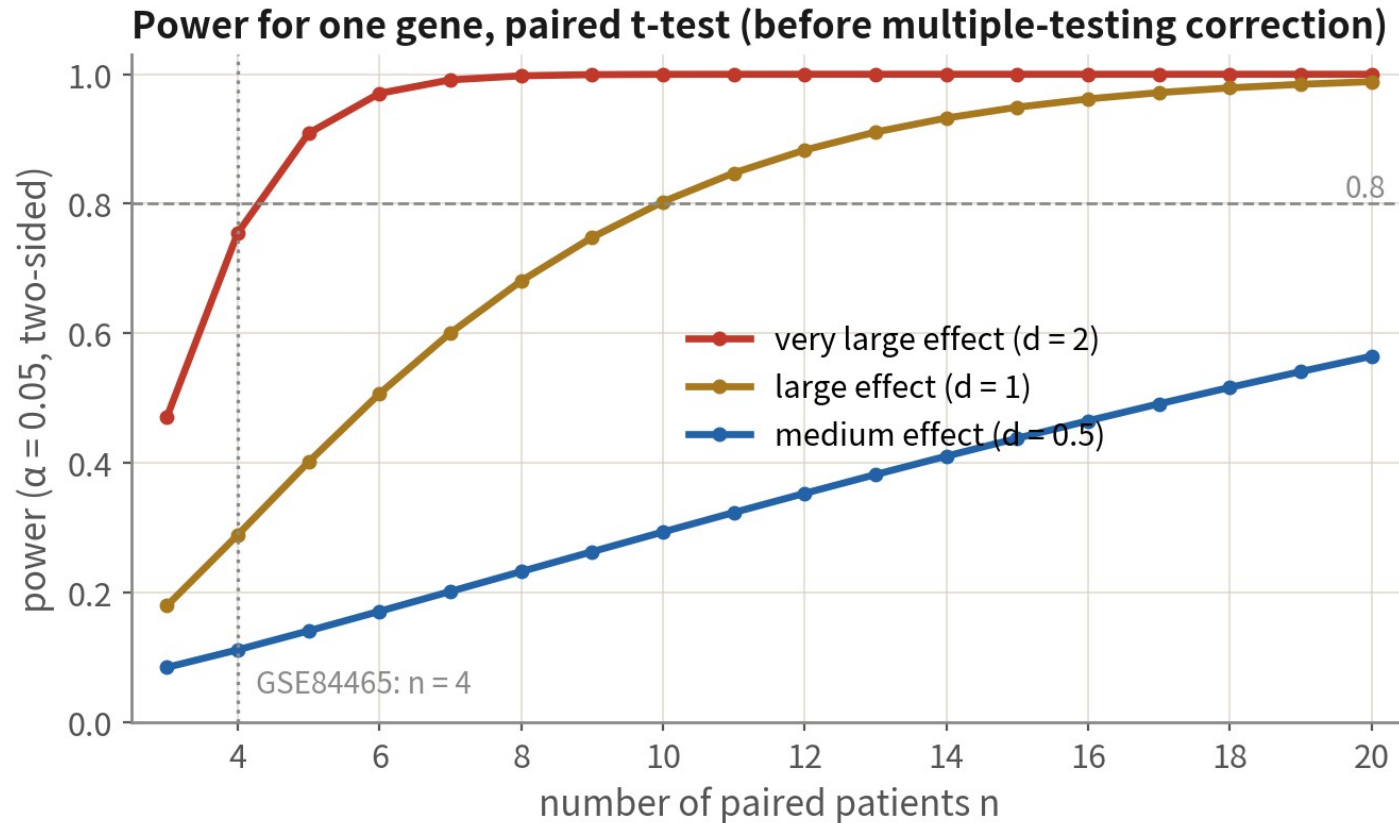
Deliverables: per-type DE tables, volcanoes, GSEA and ORA summaries

```
for (ty in names(de))                # one table per cell type
  write.csv(de[[ty]]$res, sprintf("output/tables/06_de_%s.csv", ty), row.names = FALSE)
write.csv(de.summary, "output/tables/06_de_summary_by_type.csv", row.names = FALSE)

vol<-lapply(de, function(d) {        # red = up in core, blue = up in periphery, grey = NS
  kv<-with(d$res, ifelse(padj< 0.05 & log2FC > 1, "#D62728",
    ifelse(padj< 0.05 & log2FC < -1, "#1F77B4", "grey70")))
  names(kv)<-c("#D62728"="Up in core", "#1F77B4"="Up in periphery", grey70="NS")[kv]
  EnhancedVolcano(d$res, lab = d$res$gene, x = "log2FC", y = "padj", colCustom = kv,
    pCutoff= 0.05, FCcutoff= 1, title = paste0(d$type, ": core vs periphery")) })
ggsave("output/figs/06_volcano_all_types.png", wrap_plots(vol, ncol= 2), bg = "white")
gsea.all[, leadingEdge := sapply(leadingEdge, paste, collapse = ";")] # turn the list column into a string
write.csv(gsea.all, "output/tables/06_gsea_all_types.csv", row.names = FALSE)
# NES heatmap (rows = pathway, columns = cell type) and the ORA dotplot: see script06a section 4, plots A-C
foc<-de[["Immune cell"]]             # the only type that clears the bar here
plotEnrichment(gene.sets$Hallmark[["HALLMARK_HYPOXIA"]], rank_stat(foc$res))
```

Tables carry raw and shrunk LFC, padj and baseMean; the volcano plots padj, not p; the GSEA table includes the leading edge. Everything regenerates from the script; nothing in output is hand-made | Script 06a \$5

How many patients are enough? Power comes from patients



How to read this

- 1 n=4 catches only very large effects; medium effects have <20% power
- 2 add multiple-testing over thousands of genes and fewer survive FDR—a short DE list at n=4 is expected, not a mistake
- 3 to see medium effects, a paired design needs roughly ≥ 10 patients; unpaired needs more
- 4 when power is short: treat n=4 as hypothesis-generating and validate in public or bulk cohorts

n = 4 sees only very large effects, so a short DE list is expected. Treat n = 4 as hypothesis-generating and validate in public cohorts — Pitfall 3 stated positively

When pseudobulk is impossible: three lines of defence

First admit where we are



Pseudobulk needs $\geq 2-3$ biological replicates per condition. Too few patients (skipped in 06a's de.summary, like OPC here) means falling back to cell level — where cell counts inflate p, so the result can only be hypothesis-generating

Defences 1 + 2: covariate and per-patient consistency



Run Seurat's default Wilcoxon as the baseline, then MAST with patient in latent.vars (Wilcoxon cannot) — genes significant only under Wilcoxon are mostly patient effects. Then compute logFC per patient and keep one-direction genes; NA in the table = that patient has < 5 cells on one side

Defence 3: label permutation



Shuffle tissue labels within patients and rerun: real signal should far exceed the 'significant' count under permutation. If they are comparable, that is pseudoreplication talking. State method and limits on the deliverable table

Cell-level DE done properly (script 06b)

```
obj <- subset(gbm 4, cells = colnames(gbm 4)[gbm 4$type == "0 PC"])
table(obj$patient, obj$tissue)      # look at the sample structure first: only one patient has both sites
library(tibble)
idents(obj) <- "tissue"
fm <- function(...) FindMarkers(obj, ident.1 = "Tum or", ident.2 = "Periphery",
                                logfc.threshold = 0.25, min.pct = 0.1, ...) {
  tibble::rownames_to_column("gene")      # genes live in rownames: pull them out
  de.wilcox <- fm()                       # Seurat's default Wilcoxon: the baseline
  de <- fm(test.use = "MAST", latent.vars = "patient") # defence 1: patient as a covariate
  setdiff(head(de.wilcox$gene, 30), head(de$gene, 30)) # significant only under the default ~ a patient effect
  # Defence 2: per-patient logFC; keep a gene only if the direction agrees
  # (a patient with < 5 cells on one side does not count)
  # Defence 3: permute tissue within patient 20 times, compare the real number of significant genes with the null
  perm.sig <- replicate(20, {
    o <- obj; o$tissue <- ave(as.character(o$tissue), o$patient, FUN = sample)
    idents(o) <- factor(o$tissue, levels = c("Periphery", "Tum or"))
    sum(FindMarkers(o, ident.1 = "Tum or", ident.2 = "Periphery",
                    logfc.threshold = 0.25)$p_val_adj < 0.05, na.rm = TRUE)})
  de$method <- "cell-level MAST; hypothesis-generating" # state the limitation in the deliverable table
}
```

The point is not the p value — it is the three defences and honest framing. Whenever pseudobulk is possible, use 06a; 06b is the fallback when the sample structure forbids it | Script 06b

Reference: Squair JW, et al. Nat Commun. 2021;12(1):5692. doi:10.1038/s41467-021-25960-2

06

After DE: choosing the downstream route

Six roads tumour studies take; each needs the right data

Six roads and what each needs

cell communication

malignant $\leftrightarrow$ TAM ligand-receptor axes (e.g. SPP1, CSF1)

needs: reliable labels; multiple types

CellChat

malignant states & plasticity

MES/AC/OPC/NPC-like scores, cycling

needs: CNV-called malignant cells+published gene sets

AddModuleScore

trajectory

state transitions within malignant cells (not between types)
needs: a sampled continuous process; a justified root

Slingshot

CNV subclones

subclonal structure within a patient

needs: inferCNV HMM or CopyKAT

inferCNV

deconvolving bulk cohorts

use a single-cell reference on TCGA-GBM, link to survival

needs: same-tissue reference + bulk

MuSiC / BayesPrism

spatial

which malignant state sits near vessels/necrosis

needs: Visium / Xenium data

Seurat spatial

Check the 'needs' line first, then choose. Pick the wrong data and the whole road is void

07

Cell-cell communication: CellChat from run to reading

Script 07: six plots, each answering one question

Cell-cell communication (CellChat / LIANA): three preconditions

The favourite downstream analysis in tumour studies — and the most often misused

Within one sample



Ligand and receptor must come from the same site of the same patient — macrophages from patient A cannot talk to tumour cells of patient B. Run per sample, then compare

Both ends need enough cells



Default thresholds are often 10% of cells; ambient RNA fabricates ligands. Run after SoupX, and only with CNV-called malignant cells

Results are hypotheses



Ligand-receptor co-expression at the RNA level is not binding and not signalling. Write 'X drives Y' only with a second line of evidence (spatial, protein, perturbation)

What CellChat computes

CellChatDB: ligand–receptor pairs (~3,300 in human)



Secreted signalling

cytokines, growth factors:
SPP1–CD44, TGFB1–TGFBR



ECM–receptor

matrix: COL1A1–integrins, laminins



Cell–cell contact

requires adjacency:
PD-L1–PD-1, NOTCH–JAG, MHC–TCR

Multi-subunit receptors (ITGAV+ITGB1) and co-factors are encoded in the database—the difference from simply multiplying two genes

How a communication probability is computed

①

$$L_i = \text{tri}(x_{L,i}), \quad R_j = \text{tri}(x_{R,j})$$

Trimean of L and R per group: robust to outliers, and the gene must be expressed in at least 25% of the group.

②

$$P_{i \rightarrow j} = \frac{L_i R_j}{K_h + L_i R_j} \cdot \frac{1 + AG}{1 + AN}$$

Mass action, squashed to 0-1 by a Hill function. Subunits enter as a geometric mean; agonists multiply, antagonists divide.

③

$$p = \frac{\#\{P^* \geq P\}}{100}$$

Shuffle the group labels 100 times and recompute P^* . The real P must beat 95% of them. This is 'stronger than chance', not an effect size.

④

$$P_{i \rightarrow j}^{\text{path}} = \sum_{(L,R) \in \text{path}} P_{i \rightarrow j}$$

Sum the L-R pairs of one pathway to get the pathway level; sum all pathways to get the whole network.

Three output levels: L-R pair->pathway->group-to-group. Every plot that follows is a slice of one of them

The database decides which pairs are possible, expression decides which hold, permutation decides which are not chance. Three output levels: L–R pair, pathway, group-to-group

One CellChat run: Seurat object to aggregated network (once per sample)

```
library(CellChat)
run_cc <- function(obj, label= "cc_label") {
  obj$sample <- factor(paste(obj$patient, obj$tissue, sep = "_")) # v2 requires a sample column
  cc <- createCellChat(obj, group.by = label, assay = "RNA") # uses the data layer
  cc@DB <- subsetDB(CellChatDB.human, search = "Secreted Signaling")
  cc <- subsetData(cc) |> identifyOverExpressedGenes() |>
    identifyOverExpressedInteractions()
  cc <- computeCommunProb(cc, type = "trMean", # mind the capitalisation
    population.size = TRUE) # correct for cluster size
  cc <- filterCommunProb(cc, min.cells = 20) # clusters under 20 cells do not take part
  cc <- computeCommunProbPathway(cc) |> aggregateNet()
  netAnalysis_computeCentrality(cc)
}
core <- run_cc(subset(gbm4, tissue == "Tumor" & patient == "BT_S2"))
peri <- run_cc(subset(gbm4, tissue == "Periphery" & patient == "BT_S2"))
# 4 patients x 2 sites = 8 objects; run them with lapply and keep the list
```

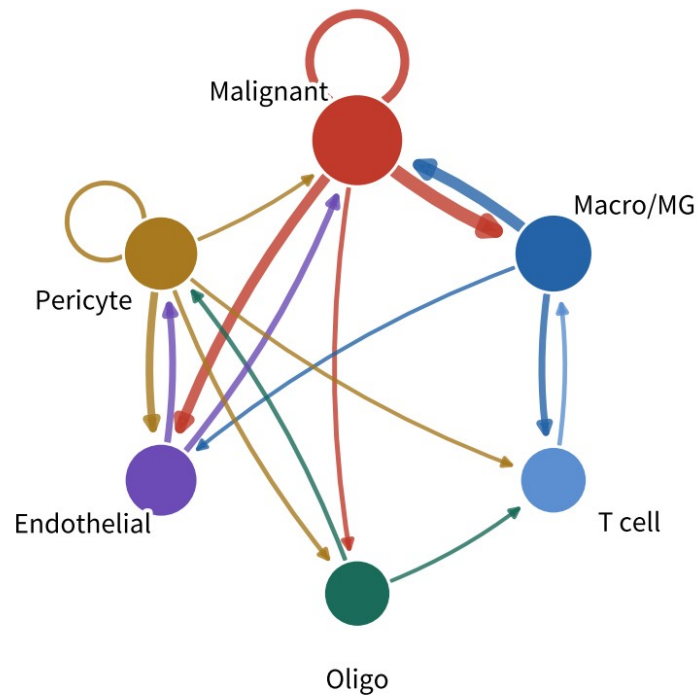
Three keys: the log-normalized data layer, one run per sample, labels after CNV calling. About 5–15 min per sample | Script 07 §1

Reference: Jin S, et al. Nat Commun. 2021;12(1):1088. doi:10.1038/s41467-021-21246-9

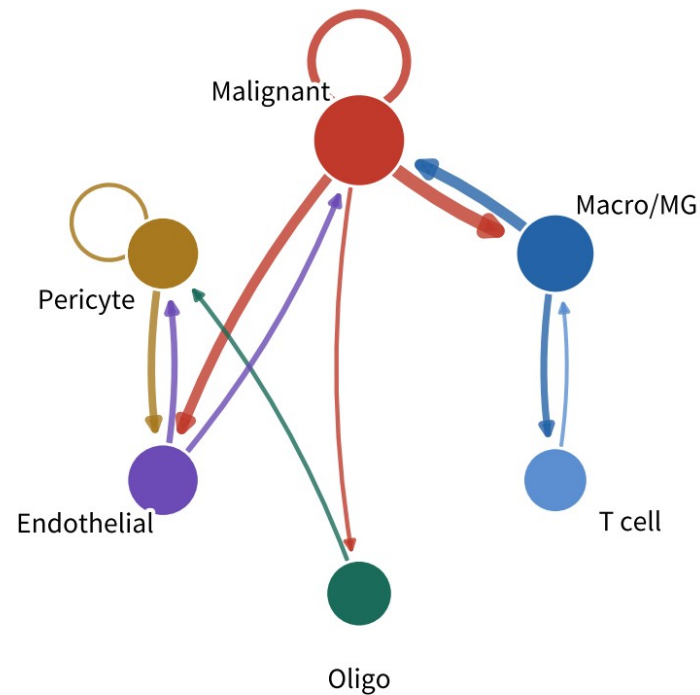
Plot 1: netVisual_circle — the whole network at a glance

schematic / simulated

Number of interactions



Interaction weights / strength



How to read

One node per group;
node size=total outgoing

Edge colour=sender; the
arrow points at the receiver

Edge width=number of
significant L-R pairs (left)
or summed probability (right)

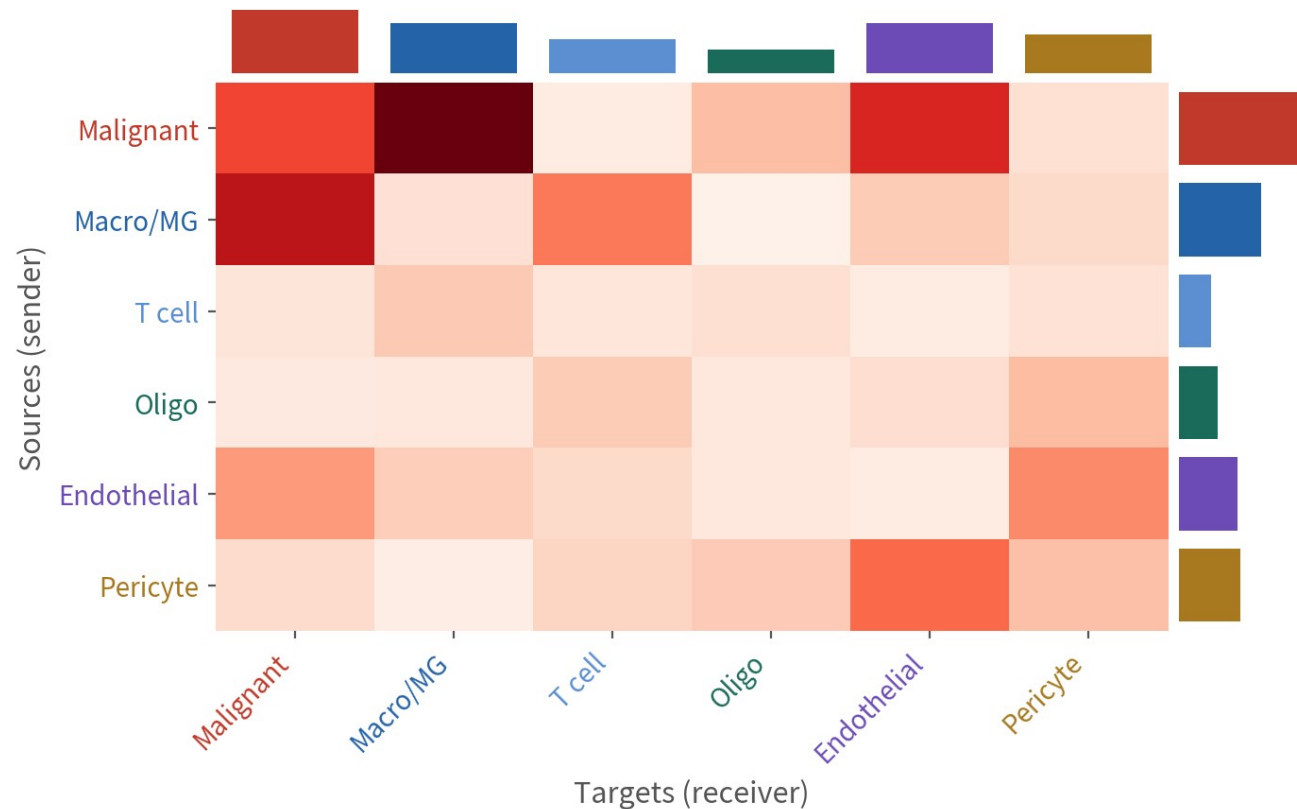
Left and right often disagree:
many pairs $\neq$ strong. Find the
main axis on the right, then
return to the left

Self-loop=within a group
(malignant autocrine)

```
groupSize <- table(cc@idents); netVisual_circle(cc@net$count, vertex.weight =  
groupSize), and again with cc@net$weight. Find the main axis on the right first
```

Plot 2: netVisual_heatmap — who sends to whom

schematic / simulated



How to read

Rows=senders, columns= receivers; darker=stronger

Right bars=row sums (top senders); top bars=column sums (top receivers)

Typical GBM: malignant↔macrophage is the brightest pair; malignant→endothelial (VEGF) is often bright too

Better than the circle plot for values; the circle plot is better for direction

`netVisual_heatmap(cc, measure = "weight", color.heatmap = "Reds")`. Rows are sources, columns targets; bars are sums. Quote numbers from this one, read direction from the circle

Pathway level: rank first, then take one pathway apart

```
cc@ netP$pathways          # the list of significant pathways
netAnalysis_signalingRole_heatmap(cc, pattern = "outgoing", height = 8) # plot4, left
netAnalysis_signalingRole_heatmap(cc, pattern = "incoming", height = 8) # plot4, right
netAnalysis_signalingRole_scatter(cc) # plot4, scatter

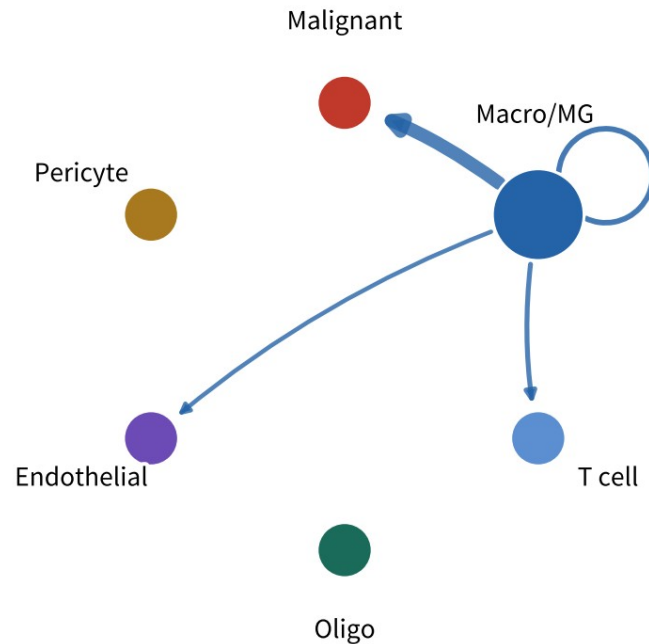
pw <- "SPP1"
netVisual_aggregate(cc, signaling = pw, layout = "circle") # plot3, left (also "chord", "hierarchy")
netAnalysis_contribution(cc, signaling = pw) # plot3, right: which L-R pair carries the pathway
netVisual_heatmap(cc, signaling = pw, colorheatmap = "Reds") # source x target for one pathway
plotGeneExpression(cc, signaling = pw) # violins of the pathway genes: confirm it
netAnalysis_signalingRole_network(cc, signaling = pw) # sender/receiver/mediator/influencer
```

Reading order: signalingRole for the actors → aggregate for direction → contribution for the concrete pair → plotGeneExpression to verify. Pathway names from cc@netP\$pathways | Script 07 §2

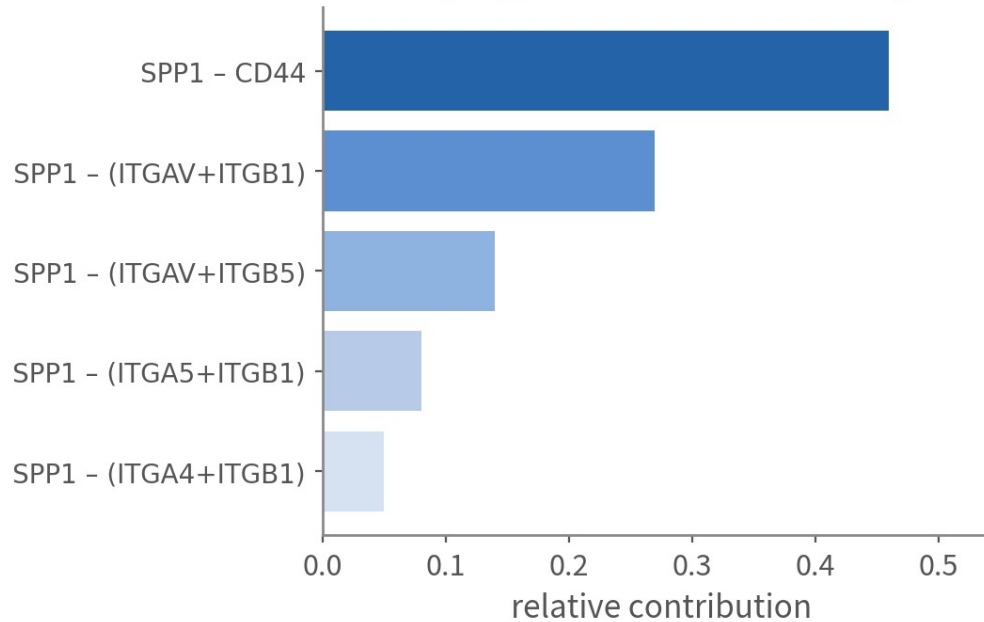
Plot 3: direction and composition of one pathway

SPP1 pathway: netVisual_aggregate (circle)

schematic / simulated



netAnalysis_contribution: which L-R pair carries the pathway

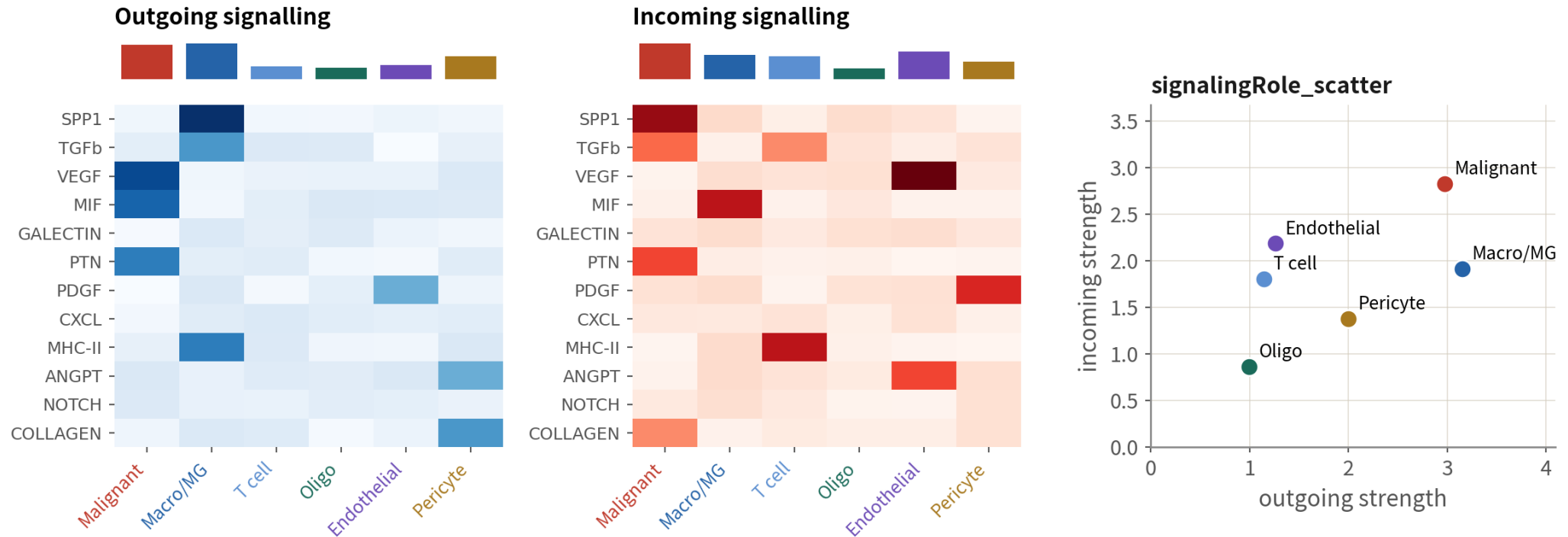


Reading: the SPP1 pathway is sent almost entirely by macrophages (blue edges) and received by malignant cells; SPP1-CD44 carries nearly half. Next: FeaturePlot and violin of that pair to confirm expression

SPP1 is the textbook TAM → malignant signal in GBM. Seeing this, the next step is not the paper — it is the violin of SPP1 and CD44 to confirm expression and fraction

Plot 4: signalingRole — each type's role on each pathway

schematic / simulated

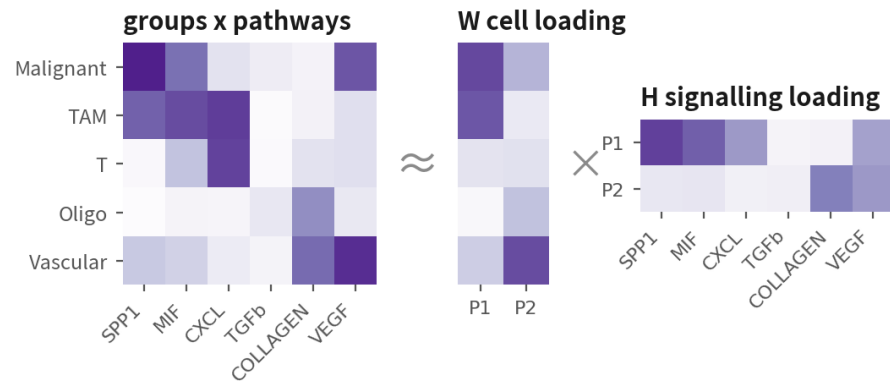


Reading: one row per pathway, one column per type; left finds senders, right finds receivers. The top-right of the scatter is the hub that both sends and receives (in GBM usually malignant cells and macrophages)

The most informative plot in the analysis: malignant cells send VEGF / MIF / PTN, macrophages send SPP1 / TGFb / MHC-II, at one glance. The top-right of the scatter is the hub

Thirty pathways is too many: communication patterns and pathway grouping

1 Communication patterns: compress 'who sends what' into a few patterns (NMF)



Pattern P1

Malignant and TAM send SPP1 and MIF together

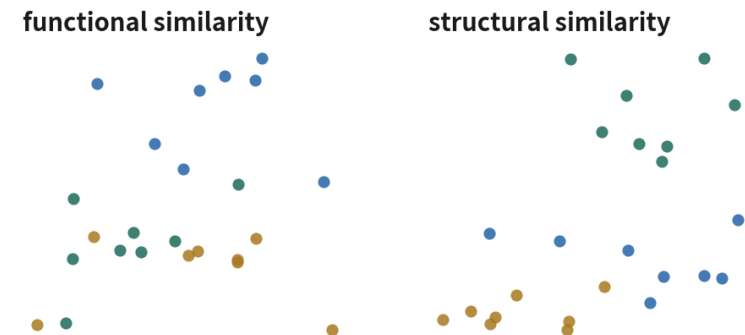
Pattern P2

Vascular and stroma send COLLAGEN and VEGF

Why: thirty significant pathways listed one by one is unreadable. Compressed into two or three patterns, we can finally say what this tumour's signalling looks like.

2 Grouping pathways: functional vs structural similarity

schematic / simulated



Functionally similar: nearly the same senders and receivers-overlapping roles, possibly redundant, and they can be written up as one statement.

Structurally similar: the same position in the network topology (one-to-many, or many-to-one), but the cells involved can be completely different. Read the two plots separately.

identifyCommunicationPatterns() uses NMF to compress groups x pathways into two or three patterns; netEmbedding + netClustering then groups pathways by function or by structure. Both exist so the conclusion is sayable | Script 07 §5 (advanced; not in the script)

Down to concrete ligand–receptor pairs, then compare two conditions

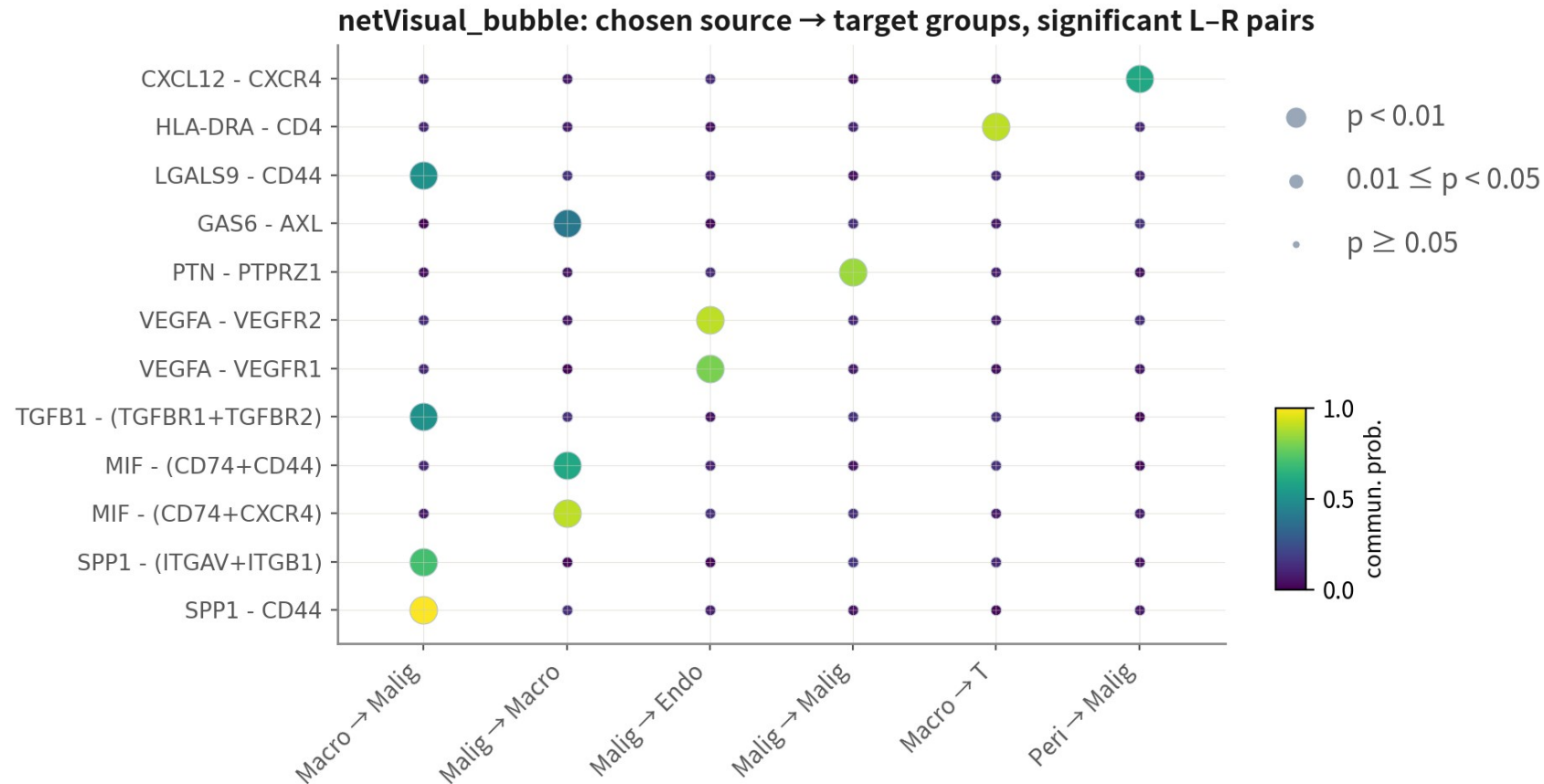
```
# Plot5: name the source and target groups, list the significant L-R pairs
netVisual_bubble(cc, sources.use = c("Macrom G", "Malignant"),
  targets.use = c("Malignant", "Macrom G", "Vascular"),
  remove.isolate = TRUE)
sig3 <- head(intersect(c("SPP1", "MIF", "VEGF"),
  cc@netP$pathways), 3) # names must match cc@netP$pathways
netVisual_bubble(cc, signaling = sig3) # just a few pathways

# Plot6: comparing two conditions - merge first
cc.m <- mergeCellChat(list(Core = core, Periphery = peri),
  add.names = c("Core", "Periphery"))
compareInteractions(cc.m, group = c(1, 2)) # number of interactions
compareInteractions(cc.m, group = c(1, 2), measure = "weight")
netVisual_diffInteraction(cc.m, weightscale = TRUE) # difference circle: red up, blue down
netVisual_heatmap(cc.m, measure = "weight") # difference heatmap
rankNet(cc.m, mode = "comparison", stacked = TRUE, do.stat = TRUE) # Wilcoxon
netVisual_bubble(cc.m, sources.use = "Macrom G", targets.use = "Malignant",
  comparison = c(1, 2), angle.x = 45) # the same L-R pair under both conditions
```

Each of the four patients has a pair of objects; when comparing, look for pathways consistent across all four, not one patient. The unit is still the patient | Script 07 §3–4

Figure 5: netVisual_bubble — the publication panel

schematic / simulated



How to read

One L-R pair per row; one source→target per column

Colour=probability, size=p: chase only the big and yellow

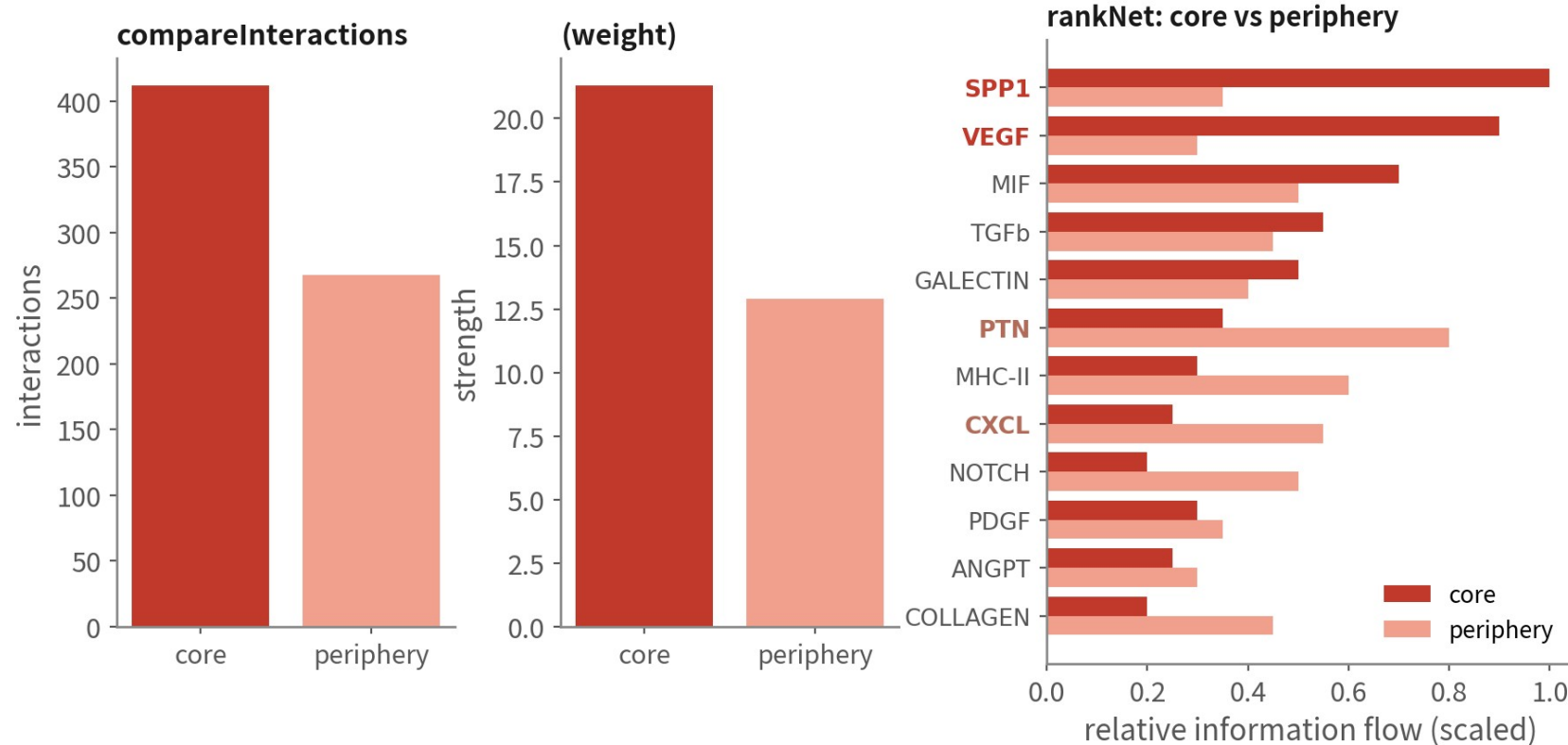
One ligand hitting several receptors (SPP1→CD44/integrins) fills several rows—they share the same ligand expression

This is the plot that goes into the paper

Chase only the big and yellow; one ligand hitting several receptors fills several rows. Once we have a concrete pair, go back to Seurat for FeaturePlot and VlnPlot

Plot 6: core vs periphery — after mergeCellChat

schematic / simulated



Reading

Run per patient, then merge and compare; bold =significant difference (Wilcoxon). Core: SPP1, VEGF (hypoxia, macrophages); periphery: PTN, MHC-II (infiltration, talking to normal brain)

The core has more and stronger interactions (hypoxia, macrophage infiltration); only bold pathways in rankNet differ between conditions. One set per patient; write it up only when all four agree

Beyond CellChat: LIANA consensus and other tools

Methods disagree substantially; reviewers often ask for agreement between at least two

Tool	Feature	Output	When
CellChat	Multi-subunit, co-factors, pathway level richest plots	L-R, pathway, network	Default; when we need the plots
LIANA+	Runs CellPhoneDB, NATMI, Connectome and more, takes a consensus rank	Consensus L-R rank	To cross-validate CellChat
CellPhoneDB	The original; curated human database	L-R + p values	Python pipelines; human data
NicheNet	Asks which ligand best explains target-cell expression change — infers from downstream genes	Ligand activity + target genes	When we have a clear receiver DE signature
Spatial validation	Visium / Xenium: are ligand and receptor actually adjacent	Co-localisation	Before writing 'X drives Y'

References: Efremova M, Vento-Tormo M, Teichmann SA, Vento-Tormo R. Nat Protoc. 2020;15(4):1484–1506. doi:10.1038/s41596-020-0292-x | Browaeys R, Saelens W, Saeys Y. Nat Methods. 2020;17(2):159–162. doi:10.1038/s41592-019-0667-5

08

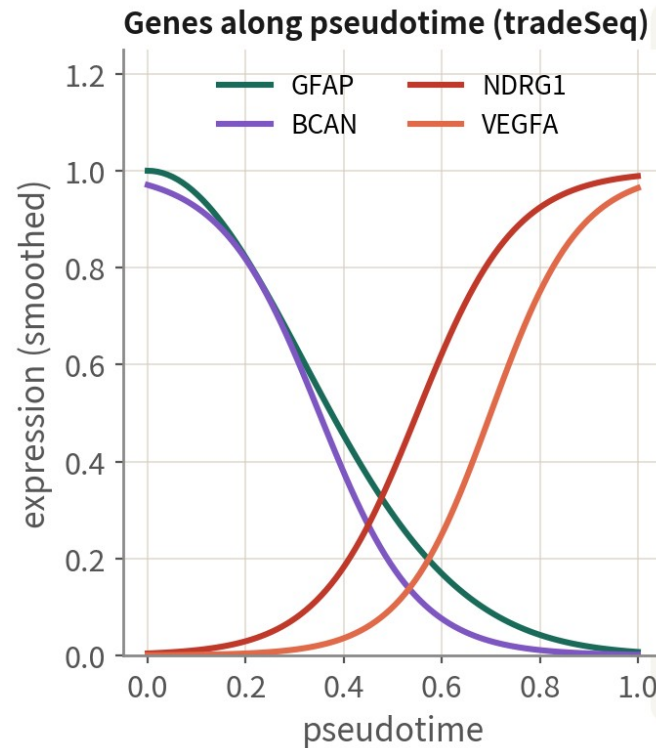
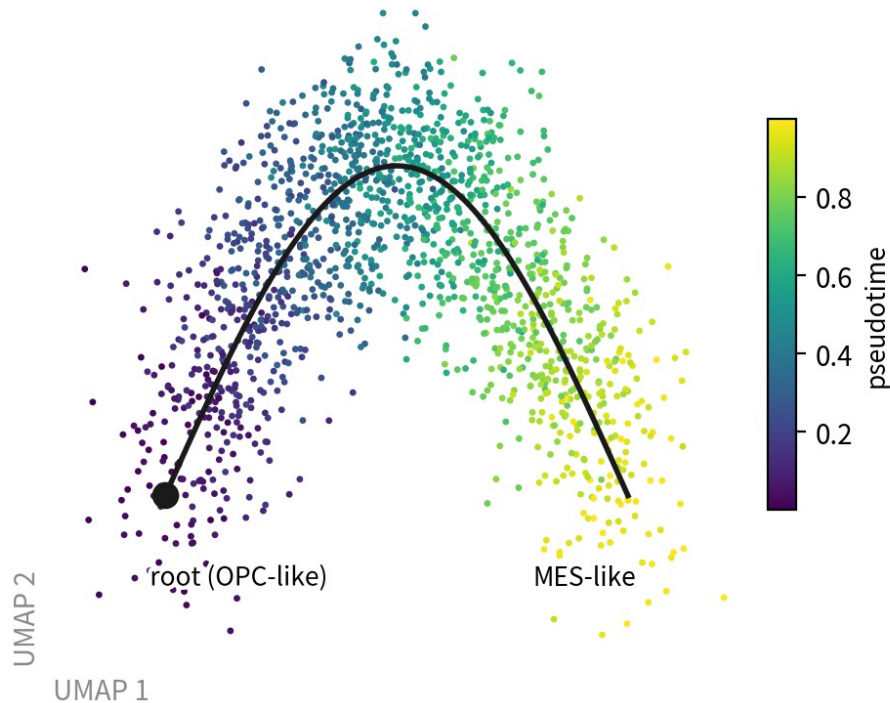
Trajectory: how states transition

Script 08: Slingshot + tradeSeq mainline; Monocle3 / Monocle2
comparison and hand-picked roots

Trajectory: reading pseudotime and gene dynamics

schematic / simulated

Slingshot: principal curve + pseudotime



Five conditions for a credible trajectory

- 1 Only within one patient's malignant cells
- 2 A biological reason for the root (OPC-like, or velocity)
- 3 Cell cycle must not drive it (else regress first)
- 4 Same direction on a new seed and other patients
- 5 The end state needs its own evidence

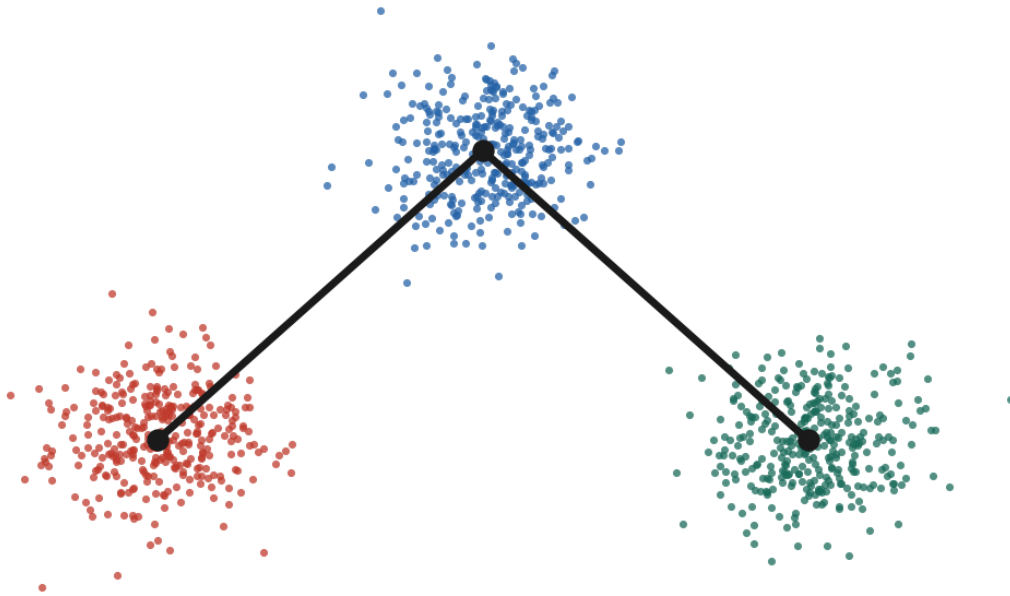
One patient's malignant cells only; the root needs a reason; the cycle must not drive it; the direction survives a new seed and other patients; the end state is verified separately. Fail any of the five and the curve is decoration

References: Street K, et al. BMC Genomics. 2018;19(1):477. doi:10.1186/s12864-018-4772-0 | Van den Berge K, et al. Nat Commun. 2020;11(1):1201. doi:10.1038/s41467-020-14766-3
| Cao J, et al. Nature. 2019;566(7745):496-502. doi:10.1038/s41586-019-0969-x

The most common downstream misuse

schematic / simulated

A 'differentiation path' malignant → immune → oligo?



A trajectory tool always draws a line

- ✗ Malignant cells do not become macrophages. The line does not exist
- ✗ Branch points at type boundaries are the classic artefact
- ✓ A sensible tumour trajectory: state transitions inside malignant cells (OPC-like → AC-like), after CNV calling, per patient
- ✓ A snapshot gives no direction; the root needs independent evidence (cycling, stemness, velocity)

The trajectory that usually holds up in a tumour is a state transition inside malignant cells - call malignancy first, split by patient first, and only then talk about pseudotime

Slingshot + tradeSeq: one patient's malignant cells

```
library(slingshot); library(tradeSeq)
m.all <- subset(gbm4, malignant == "malignant" & patient == "BT_S2") # 388 cells
m.all <- NormalizeData(m.all) |> FindVariableFeatures() |> ScaleData() |> RunPCA() |>
  FindNeighbors(dims = 1:15) |> FindClusters(resolution = 0.4) |> RunUMAP(dims = 1:15)
# Never hard-code the root cluster: pick it by 0 PC-like score, so the reason can go in the paper
m.all <- AddModuleScore(m.all, features = neftel["0 PC"], name = "0 PC")
root <- names(which.max(apply(m.all$0 PC1, Idents(m.all), mean)))
sce <- slingshot(as.SingleCellExperiment(m.all), clusterLabels = "seurat_clusters",
  reducedDim = "UMAP", start.clus = root)
m.all$pt <- slingPseudotime(sce)[,1]
# Condition 3, checked here: does the cycle drive the trajectory? |r| > 0.5 -> regress it out
m.all <- CellCycleScoring(m.all, s.features = cc.genes.updated.2019$s.genes,
  g2m.features = cc.genes.updated.2019$g2m.genes)
cor(m.all$pt, m.all$Score, use = "complete.obs") # here -0.25
cor(m.all$pt, m.all$G2M.Score, use = "complete.obs") # here -0.14, passes
# Genes changing along pseudotime (negative binomial GAM, the slowest step in this episode)
keep <- !is.na(m.all$pt)
counts <- LayerData(m.all, layer = "counts")[VariableFeatures(m.all), keep]
gam <- fitGAM(as.matrix(counts), pseudotime = m.all$pt[keep],
  cellWeights = rep(1, sum(keep)), nknots = 6)
assoc <- associationTest(gam) # 38 genes here have p underflow to 0,
head(assoc[order(-assoc$waldStat), ], 20) # so always rank by waldStat
```

The root is chosen by a score, not clicked, so it is reproducible; the cycle correlations -0.25 / -0.14 are the evidence for condition 3. fitGAM is the slowest step in this episode | Script 08 \$1

Monocle3 / Monocle2: two ways to set the root

```
# Monocle3 (the most widely used; graph-based)
library(monocle3)
cds <- SeuratWrappers::as.cell_data_set(mall) # reuses Seurat's UMAP and clustering
cds <- cluster_cells(cds) |> learn_graph(use_partition = FALSE)
# (a) interactive: a window opens and we click the root - intuitive, but not reproducible
cds <- order_cells(cds)
# (b) scripted: write the reason as code - the 10 most 0 PC-like cells as root, reproducible
root.cells <- colnames(mall)[order(mall$0_PC1, decreasing = TRUE)[1:10]]
cds <- order_cells(cds, root_cells = root.cells)
plot_cells(cds, color_cells_by = "pseudotime", label_branch_points = TRUE)

# Monocle2 (the classic DDRTree; root_state is still ours to choose)
cds2 <- reduce_dimension(cds2, method = "DDRTree") |> orderCells()
plot_cell_trajectory(cds2, color_by = "State") # read the plot, then decide which State is the root
cds2 <- orderCells(cds2, root_state = 3) # <- filled in by hand; the scripted version picks by score

cor(mall$pt, mall$pt_m2, method = "spearman") # cross-tool agreement: read it as a grade, not one cutoff
# > 0.8 strong | 0.6-0.8 needs a third, independent line of evidence | < 0.6 go back to the root
# Here r = 0.757 - the middle band; our evidence is the Nefelend-state scores and the four gene directions
```

Pick by hand to explore, script it for the final run — hand-picking is irreproducible. Read cross-tool agreement as a grade: here Slingshot vs Monocle2 gives $r = 0.757$, the middle band, carried by the end-state scores | Script 08 §2-3

References: Cao J, et al. Nature. 2019;566(7745):496-502. doi:10.1038/s41586-019-0969-x | Qiu X, et al. Nat Methods. 2017;14(10):979-982. doi:10.1038/nmeth.4402

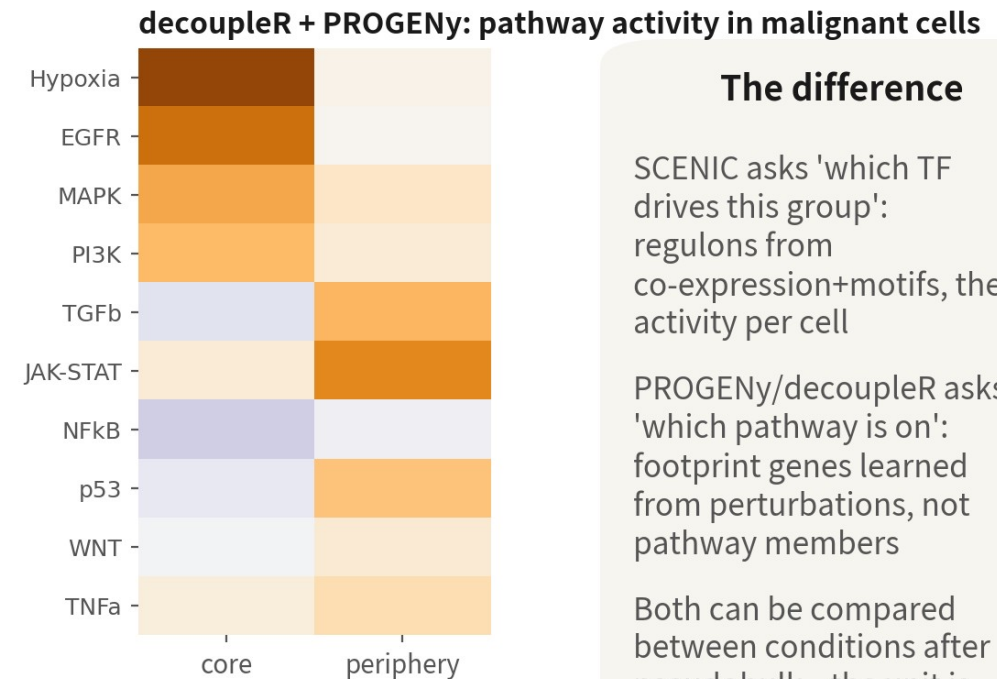
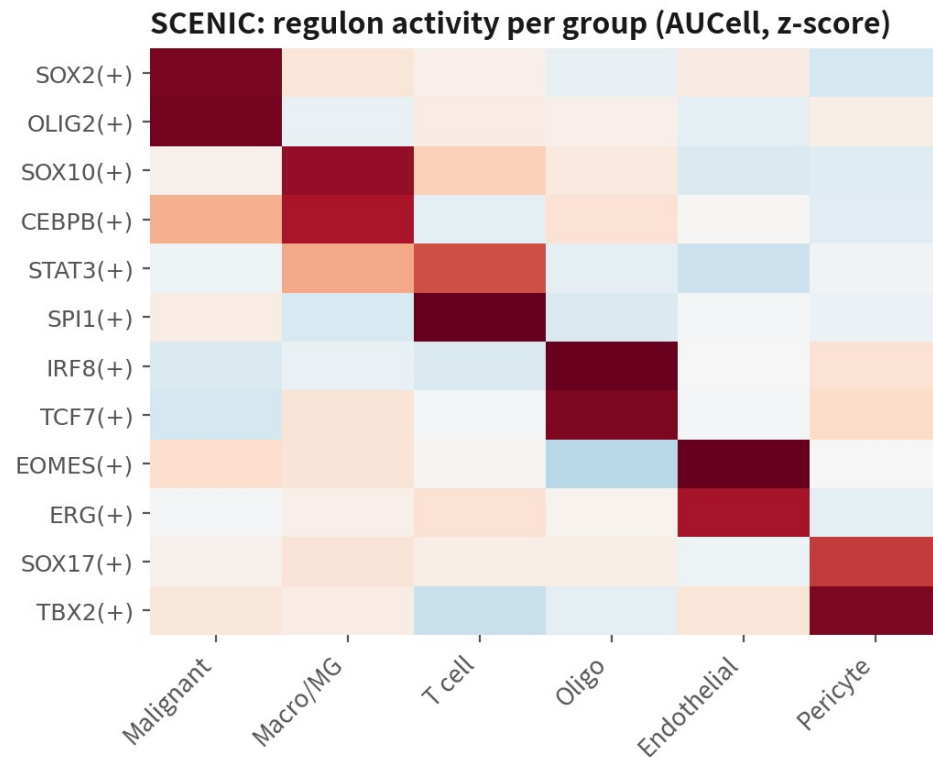
09

Pathway and TF activity: who drives it

Script 09: decoupleR / PROGENy (R, fast) and SCENIC (Python, slow, optional)

Regulation and activity: who is driving this group

schematic / simulated



The difference

SCENIC asks 'which TF drives this group':
regulons from
co-expression+motifs, then
activity per cell

PROGENy/decoupleR asks
'which pathway is on':
footprint genes learned
from perturbations, not
pathway members

Both can be compared
between conditions after
pseudobulk—the unit is
still the patient

Both panels are illustrative. Here the periphery side has just 1 / 13 / 17 / 0 malignant cells — only 3 patients pair up, and one of those values comes from a single cell. Method demonstrated; hypothesis-generating only | Script: 09

References: Aibar S, et al. Nat Methods. 2017;14(11):1083–1086. doi:10.1038/nmeth.4463 | Badia-i-Mompel P, et al. Bioinform Adv. 2022;2(1):vbac016. doi:10.1093/bioadv/vbac016 | Schubert M, et al. Nat Commun. 2018;9(1):20. doi:10.1038/s41467-017-02391-6 | Van de Sande B, et al. Nat Protoc. 2020;15(7):2247–2276. doi:10.1038/s41596-020-0336-2

decoupleR (R) and SCENIC (pySCENIC)

```
# 1. decoupleR + PROGENY: 14 pathway activities per cell
library(decoupleR)
net <- get_progeny(organism = "human", top = 500)
mat <- as_matrix(LayerData(gbm4, layer = "data"))
act <- run_mlm(mat, net, source = "source", target = "target", mode = "weight")
act.w <- tidyr::pivot_wider(act, id_cols = source, names_from = condition,
                           values_from = score) %>%
  tibble::column_to_rownames("source") %>% as_matrix()
gbm4[["progeny"]] <- CreateAssayObject(act.w)
DefaultAssay(gbm4) <- "progeny"
FeaturePlot(gbm4, features = c("Hypoxia", "JAK-STAT"))
# Comparing conditions: average activity to patient x site first,
# then a paired test - the unit is the patient

# 2. SCENIC: export aloom from R -> three pySCENIC steps -> read the AUCell matrix back
# pyscenic gm counts.loom tfs.txt -o adj.csv # co-expression (GRNBoost2)
# pyscenic ctx adj.csv *.feather --annotations_fname motifs.tbl -o reg.csv
# pyscenic auccell counts.loom reg.csv -o auc.loom # regulon activity
auc <- read.csv("output/tables/09_scenic_auc.csv", row.names = 1) # regulon x cell
gbm4[["scenic"]] <- CreateAssayObject(as_matrix(auc))
DoHeatmap(gbm4, features = c("SOX2(+)", "OLIG2(+)", "SPI1(+)", "CEBPB(+)",
                             assay = "scenic")
```

SCENIC takes hours per run; the classic GBM result is malignant SOX2 / OLIG2 and TAM SPI1 / CEBPB. The (+) after a regulon means activating | Script 09

10

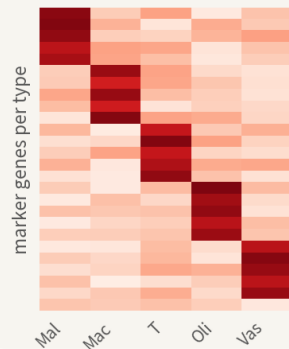
Deconvolution and survival: validation in large cohorts

Script 10: MuSiC + TCGA-GBM + KM / Cox — the exit for single-cell hypotheses

What deconvolution solves: bulk is a weighted mixture, we solve for the weights

schematic / simulated

1 Build the signature matrix S



Each column=one cell type's mean expression across genes: that type's fingerprint.

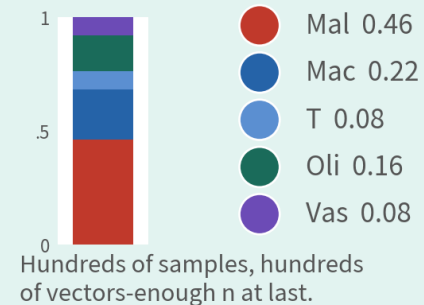
2 One bulk sample = S x proportions

$$b = S \times w$$

$\hat{w} = \arg \min_{w \geq 0} \|b - Sw\|^2$

b is this sample's expression (genes x 1), S is known, only w is unknown. Add non-negativity and sum-to-one and it is a constrained least-squares problem.

3 One set of proportions per sample



1 same tissue

A PBMC reference cannot deconvolve brain bulk

2 MuSiC's weighting

It up-weights genes consistent across subjects, reducing single-donor bias

3 relative, not absolute

w sums to 1: composition is closed, one type up means others down

4 malignant is the weak link

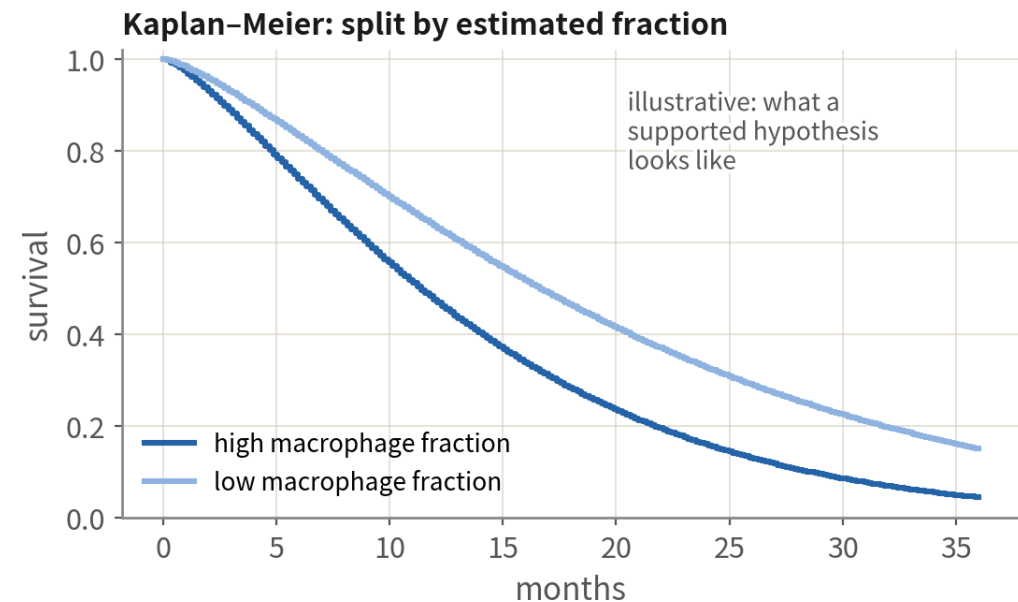
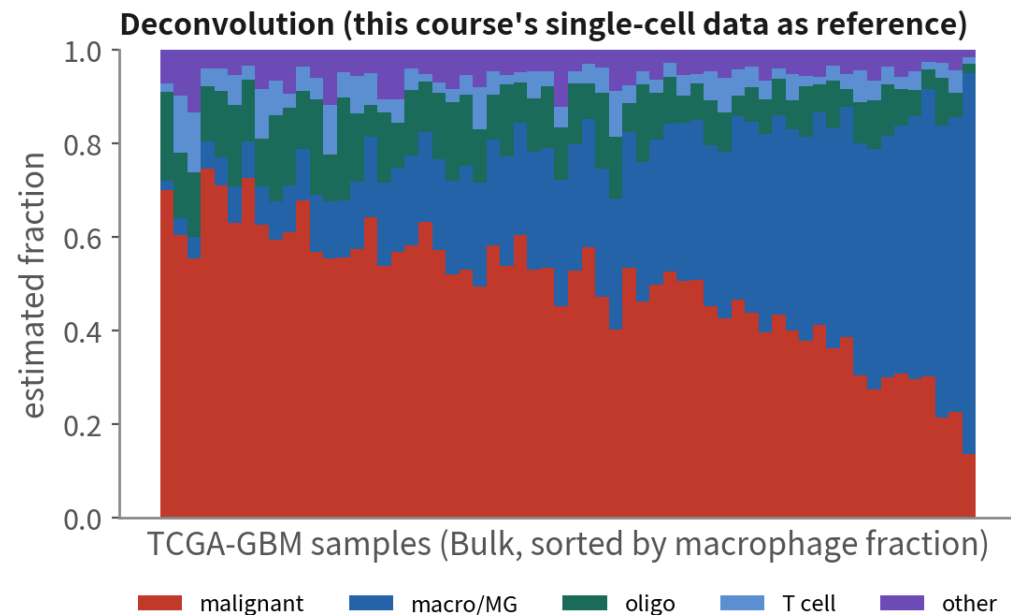
Profiles differ per patient, so one shared malignant column is an approximation

$b = S w$: the bulk expression is known, the signature matrix S built from single cells is known, only the proportions w are unknown. Add non-negativity and a sum-to-one constraint and it is solvable | Script 10

Deconvolution: taking $n = 4$ to $n = 284$

Deconvolution: carry single-cell type signatures back to hundreds of bulk samples, and buy statistical power

schematic / simulated



The right exit from $n=4$: generate the hypothesis in single cell, validate it in a public cohort. Both panels are illustrative—our own TCGA-GBM run (284 patients after de-duplication, 229 analysable):
macrophage fraction HR 1.10 (0.85-1.44), log-rank $p=0.5$; age HR 1.03 (1.02-1.04), $p=1.8e-06$

The figure is illustrative. Our TCGA-GBM run: 391 files $\rightarrow$ 372 primaries $\rightarrow$ 284 patients after de-duplication. Macrophage fraction HR 1.10 (0.85-1.44), $p = 0.46$ — no association; age HR 1.03, $p = 1.8e-06$ — the positive control does come through | Script: 10

MuSiC deconvolution + TCGA survival

```
library(MuSiC); library(TCGAbioLinks); library(survival); library(survminer)
ref<-as.SingleCellExperiment(binLayers(gbm4)); ct<-gbm4$celltype_author
ref$celltype_l1<-ifelse(gbm4$malignant=="malignant", "Malignant", # deconvolve on major groups
  ifelse(ct=="Immune cell", "Immune", ifelse(ct=="Oligodendrocyte", "Oligo",
    ifelse(ct=="Vascular", "Vascular", "Other"))))
q<-GDCquery("TCGA-GBM", data.category="Transcriptome Profiling",
  data.type="Gene Expression Quantification", workflow.type="STAR-Counts")
GDCdownload(q, directory="C:/GDCdata"); bulk<-GDCprepare(q, directory="C:/GDCdata")
mtx<-assay(bulk, "unstranded"); rownames(mtx)<-rowData(bulk)$gene_name
# * barcode field 4:01 = primary tumor; one patient often has several aliquots - de-duplicate
sel<-which(substr(colnames(mtx), 14, 15)=="01")
sel<-sel[!duplicated(substr(colnames(mtx)[sel], 1, 12))] # 391 -> 372 -> 284 patients
est<-music_prop(bulk[mtx=mtx[, sel], sc.sce=ref,
  clusters="celltype_l1", samples="patient")
prop<-as.data.frame(est$Est.prop.weighted); cl<-as.data.frame(colData(bulk))[rownames(prop),]
cl$time<-ifelse(cl$vitl_status=="Dead", cl$days_to_death, cl$days_to_last_follow_up)/30.4
cl$event<-as.integer(cl$vitl_status=="Dead") # build the time axis yourself
cl$mac_hi<-prop$Immune>median(prop$Immune)
ggsurvplot(survfit(Surv(time, event)~mac_hi, data=cl), pval=TRUE)
coxph(Surv(time, event)~mac_hi+age_at_index, data=cl) # age = the positive control
```

BayesPrism and CIBERSORTx are alternatives. Adjust the Cox model for age, MGMT and other known factors; fractions are estimates, not truth | Script 10

References: Wang X, Park J, Susztak K, Zhang NR, Li M. Nat Commun. 2019;10(1):380. doi:10.1038/s41467-018-08023-x | Brennan CW, et al. Cell. 2013;155(2):462-477. doi:10.1016/j.cell.2013.09.034

Downstream toolbox: question → tool → input → minimum evidence

Question	Tool	Input	Minimum evidence before the paper
Who talks to whom	CellChat, LIANA	Same sample, post-CNV labels, log-normalized	Two methods agree, all patients agree expression verified
How states transition	Slingshot, Monocle3 scVelo	One patient's malignant cells	Justified root + consistent across seeds / patients
Who drives it	SCENIC, decoupleR	counts (SCENIC) / data (decoupleR)	Regulon targets also move in the DE
Does it hold in a large cohort	MuSiC, BayesPrism	Single-cell reference + bulk counts	Still significant after adjusting known factors
Where in space	RCTD, cell2location	Single-cell reference + Visium	Ligand and receptor truly adjacent in space

The pitfalls, in one table

Ep.	Trap	One-line fix
Q1	Cells as samples	n is patients; pseudobulk for DE
Q1	Malignancy from markers	Markers give lineage; malignancy needs CNV and corroborating evidence
Q1	Four malignant clusters as four subtypes	Those are four patients; read states within patients
Q2	Copying the 5% mito cut	median + 3×MAD, number in the methods
Q2	Doublet cluster as immune-like tumour	Per-cluster doublet rate after clustering; remove the cluster
Q3	Integration erasing patient-specific malignant biology	Integrated space for normal cells only; malignant in unintegrated
Q3	inferCNV reference contaminated by malignant cells	Reference = immune + oligo; never astrocytes
Q3	Pooling patients for CellChat	Run per sample; report only what all patients agree on
Q3	Trajectories across patients' malignant cells	One patient, after CNV, a justified root

09

COMMON PITFALLS

Six traps that get a manuscript bounced

Pitfall 1: the wrong inferCNV reference



What it is

Using the 'Astrocyte' cluster as the normal reference because its markers look like normal astrocytes.

Why it happens

Astrocytes are normal brain cells in the textbook.

What it costs you

Most GBM malignant cells are AC-like; with malignant cells in the reference, the chr7/chr10 signal is subtracted away, the heatmap flattens and every call fails.

Pitfall 2: DE or CNV on integrated values



What it is

Running FindMarkers on core vs periphery right after integration, or feeding integrated values to inferCNV.

Why it happens

The integrated object looks like clean data.

What it costs you

Corrected values are not expression. DE returns to RNA counts and pseudobulk; inferCNV wants raw counts. One check: is the matrix integers?

Pitfall 3: clinical conclusions from $n = 4$ pairs



What it is

'Periphery malignant cells express gene X — a therapeutic target for infiltration.'

Why it happens

Paired design, pseudobulk and GSEA all done right, and p looks good.

What it costs you

Right method, insufficient power and generalisability. Four patients give candidates and a consistent direction; conclusions need more patients and independent validation (TCGA deconvolution, spatial data).

Pitfall 4: no unintegrated baseline



What it is

Running IntegrateLayers straight away and only ever looking at the integrated UMAP.

Why it happens

The integrated plot looks better, and four patients blended together looks like success.

What it costs you

Without a baseline there is nothing to compare against: you cannot separate normal cells that should mix from malignant cells that were forced to. Script 04 runs the unintegrated pass first precisely to keep that reference — every positive and negative control is built on it.

Pitfall 5: using the whole genome as the ORA background



What it is

Calling `enrichGO` without `universe`, so the whole of `org.Hs.eg.db` becomes the background.

Why it happens

It runs without `universe`, and the term list comes out longer and more significant.

What it costs you

The background should be the genes that were actually tested. Using the whole genome puts genes that are neither expressed nor tested into the denominator, inflating the ratio and turning everything significant. Pass the gene list from the DE table as `universe`.

Pitfall 6: one CellChat object for both conditions



What it is

Putting core and periphery cells into one object, running CellChat once, then splitting the plots by condition.

Why it happens

One run is faster, and the plots still render.

What it costs you

CellChat's probabilities are computed under the cell composition of that one object, so a mixed object describes neither condition. Run it once per sample and compare with mergeCellChat — which is also why the comparison is read from rankNet rather than by eyeballing two circle plots.

Q3 checklist



unintegrated plot and
object saved first



integrated space used only
to annotate normal types



inferCNV reference=
immune+oligodendrocytes



malignant labels triangulated;
unsure=unresolved



pseudobulk matrix integer,
one sample per column



DESeq2 design includes
patient (paired)



per-sample dots confirm
consistent direction



GSEA passes the
known-answer check



data prerequisites for
the next road checked

All nine boxes before it goes into a paper

ONE-LINE SUMMARY

Integration aligns the cell types shared across samples; malignant cells carry patient-specific genomic changes and are analysed per patient; the unit of differential expression is the patient; each downstream analysis has its own data requirements.

With the six sentences from Q1 and Q2, that is the skeleton of tumour single-cell analysis.

Check yourself 1

Which group is the safest inferCNV reference?

- A The GFAP-expressing astrocyte cluster
- B Immune cells and oligodendrocytes
- C Whatever sits farthest from malignant cells on the UMAP

Answer B. The reference must be certainly CNV-free. Immune cells and oligodendrocytes are essentially never malignant in GBM; astrocytes may be AC-like malignant cells; UMAP distance is not evidence.

Check yourself 2

After integrating four patients, malignant cells are one blended cluster. This means?

- A Integration worked; proceed downstream
- B Integration also erased the patient-specific CNV biology; analyse malignant cells in the unintegrated space
- C The four patients share a subtype

Answer B. Malignant cells cluster by patient largely because of genomic differences such as CNV, not batch. Cells in the same state are expected to mix across patients, so partial overlap is normal - one single blended blob is over-correction. Use the integrated space only to annotate normal cells.

Check yourself 3

For core vs periphery malignant DE, what should the DESeq2 design be?

A ~ tissue

B ~ patient + tissue

C ~ tissue + n_cells

Answer B. Each patient contributes both sites — a paired design; absorb patient first, then test site. Cell number is not a covariate; pseudobulk already handled it.

Check yourself 4

After integration the normal cells from all four patients mix well - and so do the malignant cells. What is the reasonable read?

- A Integration succeeded; carry on
- B Likely over-corrected; go back and check with the negative control and known markers
- C Malignant cells should mix — they are all GBM

Answer B. Malignant cells carry their own CNVs, so a substantial part of them separates by patient. Four patients blending completely usually means the correction removed real signal along with the patient effect. That is what the negative control is for; then confirm with a few known markers that expression itself was not altered.

Check yourself 5

A cell type has only 2 patients with a usable pair, and DESeq2 returns 300 genes at $\text{padj} < 0.05$. Report them?

- A Yes — padj is already corrected
- B No — report n first, and call the genes candidates at most
- C Tighten the threshold and then report

Answer B. A paired design at $n = 2$ has almost no power, and three hundred hits usually means the dispersion estimate is unreliable rather than that three hundred genes truly differ. Tightening the threshold does not add power. The honest version states how many patients paired up, then labels the list as candidates.

Check yourself 6

On one DE result, ORA returns no significant terms while GSEA returns five at $p_{adj} < 0.05$. How do you read that?

A They contradict each other, so neither can be trusted

B ORA is right and GSEA is too permissive

C Normal: at small n the DE list is short and ORA has little to work with; read GSEA's leading edge and see whether it makes sense

Answer C. The two ask different questions: ORA cuts a threshold first, so a short list cannot build a significant overlap, while GSEA uses the whole ranking and is therefore more sensitive at small n . Neither is wrong — read GSEA's leading edge, see whether those genes hang together biologically, and let that decide what goes into the conclusion.

FURTHER LEARNING

Three series for going deeper



Series A • Getting Started

Concepts and interpretation: experimental design, reading the figures, and turning results into a research narrative. No coding required.



Series B • Hands-on

The complete workflow on public data: from FASTQ and Cell Ranger through the Seurat pipeline and downstream analyses, with per-episode scripts. The 'Deeper: Bx' tags in this course are the index.



Series C • The Mathematics

The mathematics and statistics behind the methods: PCA and dimensionality reduction, variance stabilisation, statistical inference (why pseudobulk is valid), and how integration works.

All series are bilingual; the full index and links are in the video description.

END OF SERIES Q

Closing: a complete skeleton for scRNA-seq analysis

Practice scripts R/00–10 and the self-test quiz accompany this course.
Next, apply the workflow to our own data — tumour or otherwise.

scRNA-seq video series • Series Q Quick Start



[https://github.com/Charlene717/
scRNA-seq-tutorial-Qseries](https://github.com/Charlene717/scRNA-seq-tutorial-Qseries)